
DEPARTMENT OF INFORMATION TECHNOLOGY AND
ELECTRICAL ENGINEERING

Spring Semester 2025

Multimodal and Sparse Recurrent Transformers for Real-Time Object Detection

Master Thesis



Sebastian Heusinger
sheusinger@ethz.ch

March 2025

Supervisors: Jakub Mandula, jakub.mandula@pbl.ee.ethz.ch
Julian Moosmann, julian.moosmann@pbl.ee.ethz.ch
Pietro Bonazzi, pietro.bonazzi@pbl.ee.ethz.ch
Dr. Michele Magno, michele.magno@pbl.ee.ethz.ch

Professor: Prof. Dr. J. Lygeros, jlygeros@ethz.ch

Abstract

Detecting small Unmanned Aerial Vehicles (UAVs) remains a major challenge for security applications, since conventional radar, RGB-based, and acoustic systems struggle with issues such as low radar cross-sections, motion blur, poor lighting, and noisy environments. Thanks to their high temporal resolution and low latency, event cameras offer a promising alternative for Small Object Detection (SOD).

This work investigates two strategies for improving drone detection under real-time constraints. First, we enhance event-based filtering by integrating the Scene Adaptive Sparse Transformer (SAST) with Light-Weight DETR, achieving a 10% AP^{small} gain over the previous work of SparseDETR. However, using this method resulted in higher inference time. Second, we propose a multimodal fusion model, SAST+RGB, which combines SAST event features with YOLOX RGB features at the PAFPN stage.

Evaluations on F-UAV-D and NeRDD show that SAST+RGB achieves the highest accuracy, outperforming baselines by up to 28.2% AP_{small} with inference times below 6 ms on embedded devices. These findings highlight the potential of multimodal fusion for fast and accurate UAV detection.

Declaration of Originality

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor. For a detailed version of the declaration of originality, please refer to Appendix B

Sebastian Heusinger,
Zurich, March 2025

Contents

List of Acronyms	x
1. Introduction	1
1.1. Motivation	1
1.2. Objective	2
1.3. Research questions	2
1.3.1. Outline	3
2. Theory / Background	4
2.1. Event-based vision	4
2.1.1. Event Data Representation	4
2.1.2. Advantage of Event Cameras	5
2.2. Voxel Grid Representation of Event Data	6
2.3. Machine learning architectures	6
2.3.1. Convolutional Neural Network (CNN)	6
2.3.2. You Only Look Once (YOLO)	7
2.3.3. Feature Pyramid Networks (FPN)	8
2.3.4. Path Aggregation Feature Pyramid Network (PAFPN)	8
2.4. Vision Transformer	9
2.4.1. Attention	9
2.4.2. Self Attention	10
2.4.3. Cross Attention	11
2.4.4. Multi-Head Attention	11
2.5. Small object detection	12
2.6. Sparse Detection Transformer	12
2.7. Recurrent Neural Networks	13
3. Related Work	15
3.1. Mean Average Precision	15

Contents

3.2.	Small object detection (SOD)	15
3.2.1.	Feature hierarchies and their impact on small object detection	16
3.2.2.	Transformer-based small object detection	16
3.2.3.	Event-based SOD	17
3.2.4.	Summary SOD	17
3.3.	Sparse Networks	17
3.3.1.	Event data representations for neural networks	18
3.3.2.	Specialized architectures for continous data	18
3.4.	Filtering	18
3.4.1.	Scene Adaptive Sparse Transformer (SAST)	19
3.4.2.	SAST in relation to current event-based detectors	19
3.5.	Event-RGB Multimodal Fusion for Object Detection	19
3.6.	Summary	20
4.	Implementation	21
4.1.	Datasets overview	21
4.1.1.	Dataset characteristics	21
4.1.2.	Dataset summary	25
4.2.	Dataset integration	25
4.2.1.	Data loading	28
4.3.	Neural Network Architectures	29
4.3.1.	SAST	29
4.3.2.	Modifications to the original SAST	30
4.3.3.	SAST+LW-DETR	30
4.3.4.	SAST+RGB	33
5.	Results	35
5.1.	Models	35
5.2.	Experimental Setup	36
5.3.	Precision experiments	36
5.4.	Inference experiment	38
5.4.1.	NVIDIA GeForce RTX 4090 pytorch	38
5.4.2.	NVIDIA Jetson AGX Orin TensorRT	38
5.4.3.	Parameter Counts, FLOPs, and Resolution Effects	39
6.	Discussion	42
6.1.	F-UAV-D	42
6.2.	NeRDD	43
6.3.	Mixed	44
6.4.	Filtering models	45
6.4.1.	Effect of Pretraining on SAST+LWDETR	45
6.4.2.	Patch retention and its impact on accuracy	46
6.4.3.	Trade-offs between filtering, model size, and inference speed	46
6.5.	Multimodal SAST+RGB	47

Contents

7. Conclusion and Future Work	50
7.1. Conclusion	50
7.1.1. Filtering of event data	50
7.1.2. Multimodal approach	51
7.2. Future work	51
7.2.1. End to end pipeline	51
7.2.2. Filtering	51
7.2.3. Truncated Backpropagation Through Time (TBPTT)	52
7.2.4. Inference improvements	52
A. Task Description	53
B. Declaration of Originality	63

List of Figures

2.1. Sparse event frame	5
2.2. Histogram and voxelgrid of event data	6
2.3. Convolutional Neural Network	7
2.4. You only look once network	7
2.5. Feature Pyramid Layout	8
2.6. Path Aggregated Feature Pyramid Network	9
2.7. ViT model overview	10
2.8. Multi-Head Attention	11
2.9. Problems of small object detection	12
2.10. Recurrent Neural Network	14
4.1. Dataset bounding box size categories and width/height distributions . . .	22
4.3. Example of RGB, event data and labels from F-UAV-D	24
4.4. Example of RGB, event data and labels from NeRDD	24
4.5. Example images from the F-UAV-D dataset	24
4.6. Example images from the NeRDD dataset	24
4.7. Common dataset format	26
4.8. Dataset structure on disk after the preprocessing step.	27
4.9. SAST model architecture	29
4.10. SparseDETR: simple thresholding approach.	30
4.11. SAST+LW-DETR architecture overview.	31
4.12. Sparsity mask created from SAST.	31
4.13. SAST+LW-DETR selecting threshold	32
4.14. SAST+RGB architecture overview	34
5.1. Accuracy vs Inference time graph	40
5.2. Command used for inference tests	40
6.1. General problems with NeRDD dataset	43
6.2. NeRDD timestamp skip	44

List of Figures

6.3. Bounding box after synchronizing with 30FPS.	44
6.4. Mask comparison best case	46
6.5. Mask comparison worst case	46
6.6. Bounding boxes of YOLOX-RGB on a complex background.	48
6.7. SAST+RGB complex background low event activity	49
6.8. SAST+RGB complex background high event activity	49

List of Tables

2.1. Filtering problem of SparseDETR	13
3.1. Performance different ANNs from literature	20
3.2. NeRDD, FAOD and Ev-SpSegNet performance table	20
5.1. Overview of different tested models and their architectures	36
5.2. F-UAV-D dataset mAP comparison	37
5.3. NeRDD dataset mAP comparison	37
5.4. Mixed dataset mAP comparison	38
5.5. Inference time comparison mutiple architectures	41
6.1. Average number of patches retained.	47
6.2. Comparision SAST+RGB vs NeRDD Pooling (Encoder)	48

List of Acronyms

ANNs	Artificial Neural Network
AP	Average Precision
CNNs	Convolutional Neural Networks
DETR	DEtection TRansformer
ERGO	Event Representation through Gromov-Wasserstein Optimization
FPN	Feature Pyramid Networks
GNNs	Graph Neural Networks
GWD	Gromov-Wasserstein Discrepancy
IoU	Intersection over Union
LLMs	Large Language Models
LSTM	Long short-term memory
NLP	Natural Language Processing
NN	Neural Network
PAFPNs	Path Aggregation Feature Pyramid Networks
PBL	Center for Project-Based Learning

List of Acronyms

RNNs		Recurrent Neural Networks
RVT		Recurrent Vision Transformer
SAST		Scene Adaptive Sparse Transformer
SAST		Scene Adaptive Sparse Transformer
SNNs		Spiking Neural Networks
SOD		Small Object Detection
ViT		Vision Transformer
YOLO		You Only Look Once

Introduction

In recent years, unmanned aerial vehicles (UAVs), or drones, have become increasingly widespread [1]. Advances in technology have enabled the use of drones across a wide range of positive applications, including healthcare and emergency service [2], agriculture [3], infrastructure inspection [4], entertainment [5], and commercial logistics [6].

The increasing number of drones also raises the potential for misuse, such as airspace violations [7], [8], trespass, privacy invasion [9], espionage [10] and critical infrastructure [11].

Critical infrastructure includes power plants, airports, ports, nuclear facilities, water treatment plants, communication hubs, and data centers. These sites are vital for national security, stability, and economic well being. Even a single drone intrusion, whether intentional or accidental, can cause major disruption. It may damage assets, interrupt essential services or put lives at risk. Public spaces are also vulnerable. Large gatherings, sports events, government buildings, and city centers remain sensitive environments where drones can pose challenges to safety and security.

Because of these risks, drone detection is a vital part of security strategies. Detecting drones early, before any harm is done, allows authorities to monitor, intercept, or mitigate threats.

1.1. Motivation

Despite the urgent need for reliable drone detection, many existing technologies face important limitations. Radar-based systems, while widely used in military and air-traffic surveillance, are often ill-suited for small drones. Small UAVs present very low radar cross-sections, making detection difficult, and continuous radar operation is associated

1. Introduction

with high energy costs [12]. Acoustic-based systems [13], on the other hand, exploit the characteristic sound signatures of drone propellers. They are lightweight and energy-efficient, but their performance degrades sharply in noisy environments such as cities, industrial zones, or near airports.

Given these challenges, RGB cameras can be an attractive alternative. Equipped with a zoom lens, they can scan the sky and detect far-away objects. Robust machine learning algorithms [14], [15], [16] for object detection are already available and can be applied to this setting.

However, RGB cameras have inherent limitations in poor lighting and are prone to motion blur, which is a critical issue for fast moving small objects like drones. Event cameras [17] can be a valuable addition. They provide high temporal resolution, low latency, and strong performance in low light environments. This makes them well suited to operate as a standalone modality, or to complement RGB cameras in a multimodal setup.

1.2. Objective

The objective of this work is to develop a fast detection model that can be deployed on edge devices with a good accuracy for Small Object Detection (SOD). Because drones are fast moving and scanning larger areas of the sky is directly tied to algorithmic speed, a fast detection model is essential.

In the first approach, we aim to improve SOD accuracy by enhancing the filtering of sparse event data based on the method proposed in [18], while keeping inference time low.

In the second approach, we investigate a multimodal strategy that combines event and RGB data to further increase accuracy. Here, the focus is on evaluating how much SOD performance can be improved when accepting a higher inference cost due to the additional computational demand of using both modalities.

1.3. Research questions

- How can filtering strategies be improved to enhance small object detection in event-based vision while maintaining low inference cost?
- To what extent does incorporating RGB data alongside event data improve the detection of small objects compared to using event data alone?
- Can expanding the training dataset lead to more accurate and robust small object detection?

1. Introduction

- Can multimodal neural networks be designed to leverage both RGB and event data to further improve small object detection accuracy?

1.3.1. Outline

The thesis is structured into 7 sections. In section 2, the theory necessary for the later discussion is covered, going over current object detection networks and the approach of filtering patches for sparse transformers. Section 3 reviews related work on object detection and multimodal approaches, including the Scene Adaptive Sparse Transformer (SAST) network and the transformer based LW-DETR network. In Section 4, we analyze the datasets, describe the preprocessing procedures, and present the implementation of our neural network architectures. Section 5 presents the results of baseline models such as YOLOX, LW-DETR, and SAST, along with our implementations of more advanced models. The discussion of the results is provided in Section 6, followed by the conclusion and outlook in Section 7.

Chapter 2

Theory / Background

This chapter provides the relevant theory and background to understand the main concepts necessary for this work. At first event-based vision is explained and the advantages as well as how this data is formatted into a voxel grid to be further processed by neural networks.

2.1. Event-based vision

Within the field of computer vision, the present work focuses on event-based vision. Unlike traditional cameras, which generate sequential images at fixed intervals, event-based cameras produce a continuous stream of data that encodes variations in pixel brightness. These cameras respond only to changes in brightness, transmitting them asynchronously and independently at the pixel locations where they occur.

2.1.1. Event Data Representation

The output of the event camera, E_n , is comprised of single events or spikes that occur in the event sensor. It is represented as

$$e_n := (x, y, t, p),$$

where (x, y) is the location of the event occurring on the sensor, t is the timestamp, and p is the polarity. When the brightness of the pixel increases and decreases, the polarity values are +1 and -1 respectively. Because events are only recorded at the specific positions where they occur in the frame, event data is inherently sparse. An

2. Theory / Background



Figure 2.1.: Sparse event frame, with most pixels inactive.

example of the data generated by an event-based camera is shown in fig. 2.1. Most of the event frame is inactive as event cameras only record changes.

2.1.2. Advantage of Event Cameras

Event-based vision provides clear advantages over conventional cameras. An overview of the main benefits is given below.

Asynchronous: The asynchronous operation and microsecond resolution of event cameras allow them to capture very fast motion. The high acquisition rate of these cameras also prevents motion blur in the captured data.

Low Power: Event cameras are inherently low power because they transmit only changes in brightness at the pixel level. This simpler operating principle eliminates redundant data and allows the sensor and processor to remain idle unless meaningful information needs to be captured. For reference, Prophesee’s [19] smallest camera operates at only $36\text{ }\mu\text{W}$ in ultra-low-power mode, while high-frame-rate cameras [20] operate in the range of multiple watts. Even cameras marketed as low-power fall in the range of approximately 200 mW [21], [22].

High Dynamic Range (HDR): Event cameras achieve a high dynamic range since each pixel operates asynchronously and independently. They measure relative changes in brightness rather than absolute intensity, which allows them to capture details across very dark and very bright regions simultaneously. To integrate event-based data with established vision-based

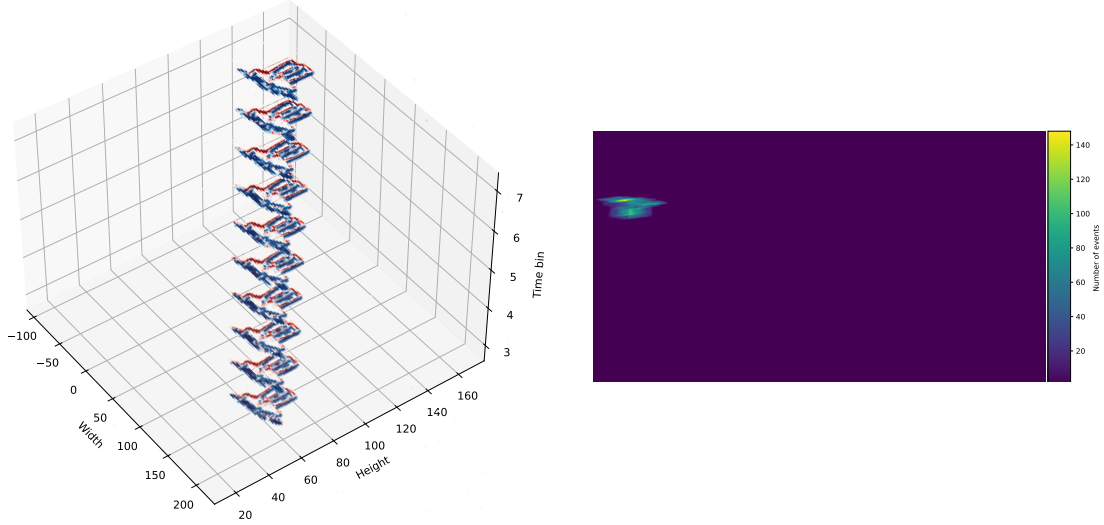


Figure 2.2.: Voxel grid (left) and 2D histogram (right) of the same scene.

2.2. Voxel Grid Representation of Event Data

To integrate event-based data with established vision-based Artificial Neural Network (ANNs), the event data needs to be converted to a processable format. Several approaches have been proposed to generate event-based input for machine learning [23]. In the present work, the **voxel grid** was used as input for all the models analyzed.

The voxel grid is generated by collecting events over a defined time window and assigning them, together with their polarity, to discrete bins. The outcome of such process is displayed in fig. 2.2. The left image in fig. 2.2 shows the voxel grid representation, which consists of a stacked polarity histogram over time also called voxel grid [23]. On the right, a simple 2D histogram is displayed to make the scene more clear. For the voxel grid we used 10 bins with a bin size of 50 ms and an additional 2 bins for the polarity. In fig. 2.2, the polarity bins are represented by red and blue. Red are the negative event activations, while blue are the positive ones. The z-axis represents the 10 time bins.

2.3. Machine learning architectures

2.3.1. Convolutional Neural Network (CNN)

Convolutional Neural Networks (CNNs) are a class of deep learning models designed to learn spatial patterns from images. Instead of looking at the entire input at once, CNNs use filters (kernels) that slide over small regions of the image to detect features such

2. Theory / Background

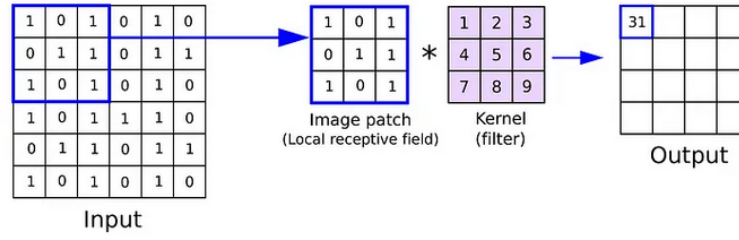


Figure 2.3.: CNN convolution explained. Image from [24].

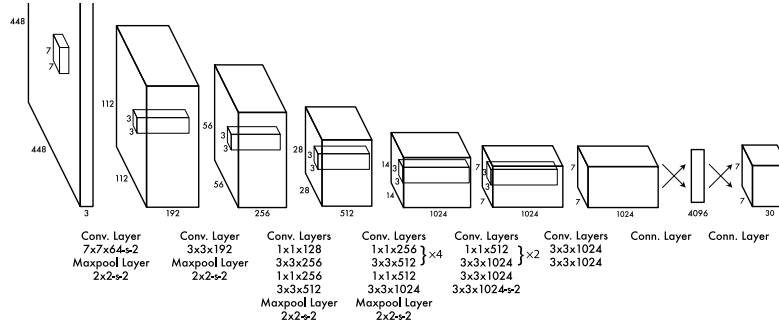


Figure 2.4.: YOLO Network taken from [25].

as edges, textures, or shapes. By stacking many layers, the network learns increasingly abstract patterns, from simple edges in early layers to complex objects in deeper layers. This hierarchical feature extraction makes CNNs especially powerful for vision tasks.

2.3.2. You Only Look Once (YOLO)

The You Only Look Once (YOLO) [25] networks are widely used network architectures in computer vision. Object detection models such as YOLO build on the idea of CNNs. In fig. 2.4, the architecture for the first YOLO network is displayed. They combine CNN-based feature extraction with prediction heads that not only classify objects but also localize them in an image using bounding boxes. This design enables real-time detection. They introduce the novel idea of doing detection and classification in one single forward pass from a single input image. Unlike traditional detection methods that scan an image in multiple stages, YOLO divides the image into a grid and simultaneously predicts multiple objects per grid cell, making it fast and suitable for real-time applications. One of the most recent contributions in this line of research is YOLOv11 [15].

2. Theory / Background

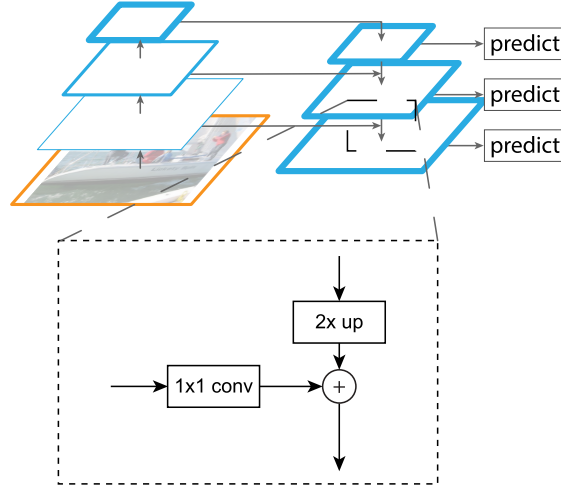


Figure 2.5.: Feature Pyramid Network taken from [26].

2.3.3. Feature Pyramid Networks (FPN)

Feature Pyramid Networks (FPN) [26] were introduced to enhance scale-invariant object detection by combining feature maps at multiple resolutions. As image input comes in different scales FPNs use a hierarchical structure to extract feature maps at different resolutions, preserving spatial detail and semantic information. In fig. 2.5, the structure of the feature pyramid is displayed. It illustrates how the features, generated by convolutional layers, get upsampled and added back to lower layers to keep spatial information. FPNs and later Path Aggregation Feature Pyramid Networks (PAFPNs) are used in multiple architectures to improve the performance of the whole architecture and are major building blocks for several YOLO models [27], [28].

2.3.4. Path Aggregation Feature Pyramid Network (PAFPN)

PAFPNs represent an extension of FPN, designed to further improve detection accuracy. In [29], they show that higher level layers correspond to detection of complete objects while lower level layers correspond to detecting edges and corners. In [30], they use this insight, and make the point that edges correspond to the transition to other objects and background. Thus lower level features are important to more accurately get bounding boxes. To include this lower level features also in lower levels with a shorter path they introduce the design shown in fig. 2.6. The green arrow indicates the shorter path to incorporate low level features into higher level ones. The red arrow has to go through 100+ layers, for introducing low-level features into the higher lever layers, while the green path only takes 10 layers. Together, the path-aggregation design and adaptive feature pooling (see fig. 2.6) produce tighter, more accurate bounding boxes.

2. Theory / Background

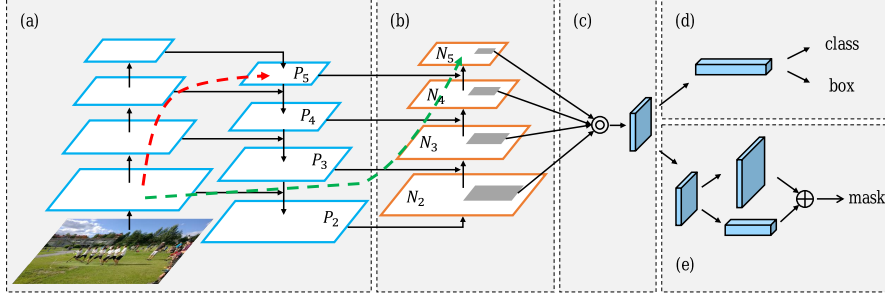


Figure 2.6.: Path Aggregated Feature Pyramid Network taken from [30].

2.4. Vision Transformer

Vision Transformers represent a recent alternative for object detection and were popularized by [31].

The Vision Transformer (ViT) builds on the transformer architecture and its attention mechanism. The transformer approach, based on Multi-Head Attention, was first introduced in Natural Language Processing (NLP) by [32] and later adapted to vision tasks by [31]. For images, the input is divided into patches that are embedded as tokens, analogous to the tokenization of text.

Transformers process sequences, such as text or image patches, in parallel rather than sequentially. The patch size determines the number of input tokens and therefore affects model size and computation time. Smaller patches can improve accuracy but also increase cost, since the complexity scales quadratically, $\mathcal{O}(N^2 \cdot D)$, with the number of inputs N and embedding dimension D . As a result, transformers are memory intensive. In [31] it is shown that a patch size of 16×16 px offers a good balance between accuracy and efficiency.

An overview of the patching, embedding, and Transformer Encoder block is shown in fig. 2.7.

The next subsections introduces the attention mechanisms, the central operations of the Transformer.

2.4.1. Attention

Attention blocks are the core of transformers. Instead of treating tokens independently, each token attends to all others through learned weights that indicate their relative importance. This mechanism is formalized by the attention function,

2. Theory / Background

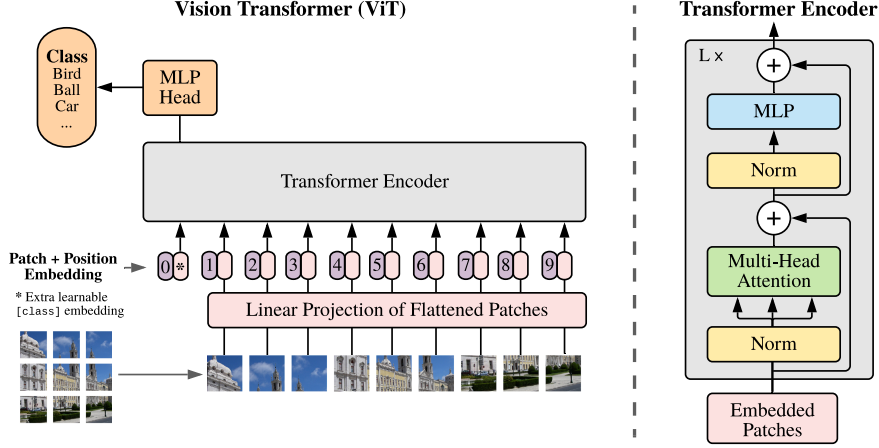


Figure 2.7.: Vision transformer model overview from [31].

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

where the queries Q , keys K , and values V are projections of the input tokens. The dot product between queries and keys determines the relevance of one token to another, the softmax normalizes these scores into attention weights, and the weighted sum with V produces context-aware token representations. In this way, the model captures long-range dependencies and enriches each token with information from the broader sequence.

2.4.2. Self Attention

For self-attention, each input token x is projected into three representations:

$$\begin{aligned} Q &= W^Q x, \\ K &= W^K x, \\ V &= W^V x, \end{aligned}$$

where W^Q , W^K and W^V are learnable projection matrices. The queries (Q) are compared with the keys (K) to determine how strongly each token should attend to others, and these attention weights are applied to the values (V). In this way, the model learns to focus on the most relevant parts of the input sequence.

2. Theory / Background

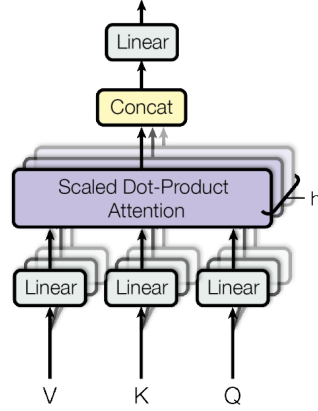


Figure 2.8.: Multi-Head Attention from [32].

2.4.3. Cross Attention

For cross-attention, the queries are obtained from one sequence of input tokens x , while the keys and values are derived from another sequence y :

$$\begin{aligned} Q &= W^Q x, \\ K &= W^K y, \\ V &= W^V y, \end{aligned}$$

where W^Q , W^K and W^V are learnable projection matrices. In this setup, the tokens in x attend to the tokens in y , allowing information to flow between different inputs. Using queries from one input and keys/values from another makes it possible to align or fuse information across inputs. For example, combining visual features with textual descriptions.

2.4.4. Multi-Head Attention

Instead of relying on a single attention block, multi-head attention performs several attention operations in parallel. The queries, keys, and values are each linearly projected h times with separate learnable parameters, producing h different attention heads. Their outputs are then concatenated and combined through a final linear projection. Figure 2.8 illustrates this arrangement, where each head is implemented using a Scaled Dot-Product Attention block.

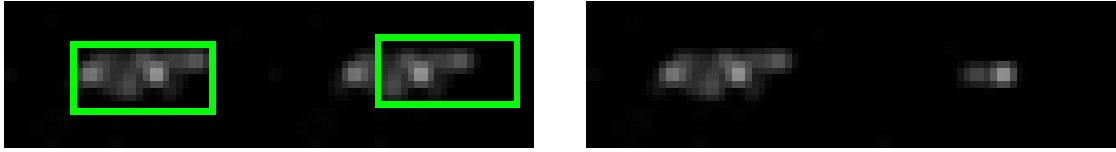


Figure 2.9.: Problems in computer vision that affect small objects more than bigger objects. On the left the problem of small misalignments from labeling and on the right the problem of down sampling images.

2.5. Small object detection

SOD remains a major challenge in computer vision, with several factors contributing to its difficulty. Small objects are typically defined as occupying fewer than 32×32 px in an image. Due to their limited pixel representation, much of their geometric information is lost in deeper network layers, which often results in false detections. They are also more easily occluded, for example when drones disappear behind trees, buildings, lamp posts, or other obstacles. Occlusion further complicates annotation, since a partially hidden small drone may become nearly indistinguishable, whereas larger, closer objects remain easier to label even when partly obscured.

Annotation quality is another issue. Many datasets rely on semi-automated labeling pipelines that use state-of-the-art ANNs to generate bounding boxes [33]. While these systems perform well for larger objects, they tend to introduce misalignments for smaller ones. Even small localization errors can disproportionately reduce accuracy as a shift of only a few pixels may have little effect on a large object but can drastically lower the intersection-over-union (IoU) for a small one, potentially omitting much of the object itself.

Finally, prevailing object detectors suffer from information loss due to subsampling operations in their backbones. Reducing the spatial resolution of feature maps disproportionately affects small objects, as much of their limited information is lost in the process. Figure 2.9 illustrates both a bounding-box misalignment and the effects of downsampling on a small object.

2.6. Sparse Detection Transformer

The previous Master’s thesis by David Zehnder [18] explored a sparse DETECTION TRANSFORMER (DETR) approach, which leverages the inherent sparsity of event data to accelerate inference while detecting drone bounding boxes. In the following, this approach is reviewed as a basis for the present work.

2. Theory / Background

F-UAV-D dataset	Number of Boxes	Percentage of Boxes
Total bounding boxes	77599	100%
Small bounding boxes	62688	80.78%
Excluded bounding boxes (all small)	2895	3.73%
Partially included bounding boxes	56582	72.92%

Table 2.1.: Overview of how many bounding boxes are filtered before reaching the network for detection. This table is for the selected threshold of 100 event activities.

SparseDETR builds on a DETR variant, which is a Vision Transformer augmented with a head for bounding-box estimation [34]. Specifically, the LW-DETR [16] serves as the base architecture.

To improve the inference speed of LW-DETR, the event data in voxel grid form is divided into patches, which are then masked to retain only regions with high event activity. Patches that exceed a predefined threshold are selected and passed to LW-DETR. Compared to the baseline LW-DETR model, this method reduces inference time by approximately 2 ms.

Threshold

The threshold for selecting patches is chosen empirically. In this work, a threshold of 100 event activities is used, striking a balance between discarding too many patches and retaining the most relevant regions

Problems with patch selection / filtering

In the sparse approach, each patch is evaluated based on its event activity before being passed to LW-DETR. Only patches that exceed a predefined threshold are selected as input while all others are discarded. This creates a particular problem for SOD, since small objects may fall within patches that do not meet the threshold. As a result, LW-DETR receives no input covering the object and therefore cannot detect it. As shown in table 2.1, setting the threshold to 100 excludes 3.73% of bounding boxes entirely. The proportion of bounding boxes that are only partially included is even higher.

2.7. Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are a class of neural networks designed to process sequential data. Unlike feed-forward networks, which treat each input independently,

2. Theory / Background

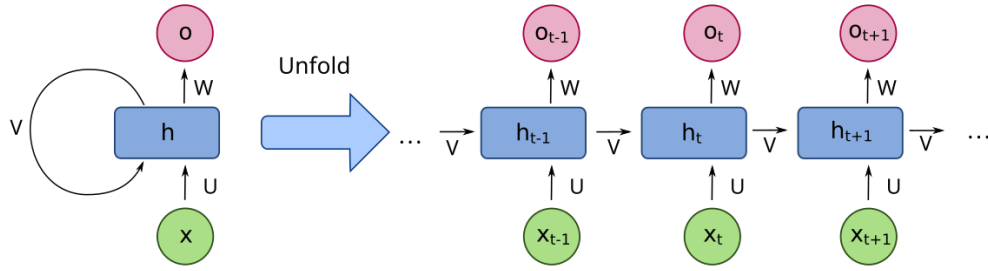


Figure 2.10.: Recurrent Neural Network overview of how a single recurrent block works. Image from [37].

RNNs maintain a hidden state that carries information from previous time steps. At each step, the network takes the current input and the hidden state from the previous step to produce an output and update its hidden state. In fig. 2.10, such block of hidden states over time is shown. Because RNNs capture temporal dependencies, they can naturally exploit the sequential characteristics of event data. However, standard RNNs struggle with very long sequences due to issues like vanishing and exploding gradients, which motivated extensions such as Long Short-Term Memory (LSTM) [35] and Gated Recurrent Unit (GRU) networks [36].

Related Work

In this chapter, we present the current state of the art for drone detection machine learning architectures. The focus of this review of the literature is on network speed, i.e., network size, as well as on small drone detection capabilities.

3.1. Mean Average Precision

Mean Average Precision (mAP) is the standard metric for evaluating object detection models. It is derived from Average Precision (AP), which measures the area under the precision-recall curve for a given class at a chosen Intersection over Union (IoU) threshold. To account for multiple classes, mAP averages AP across all categories. In the PASCAL VOC benchmark [38], AP was computed at a single IoU threshold of 0.5, whereas the COCO benchmark extended this definition by averaging AP across multiple IoU thresholds from 0.5 to 0.95 in steps of 0.05, making mAP the most widely used metric today.

COCO’s [39] metric is used in this work to evaluate models and perform comparisons. The AP^{small} metric, which measures performance on small objects, is of particular importance here. In COCO, small objects are defined as smaller than 30×30 px. Since this metric is not commonly reported in the literature, we also adopt mAP (denoted as $AP^{\text{IoU}=.50:.05:.95}$) for consistency and report both metrics to enable fair comparison.

3.2. Small object detection (SOD)

Small object detection is an active field of research. There are different fields that benefit from small object detection, from large scale imaging with satellites [40], where even

3. Related Work

large objects can appear small and the input size is a major factor in designing the network, maritime surveillance [41], to medical approaches [42], where small features in images highlight abnormalities.

3.2.1. Feature hierarchies and their impact on small object detection

In "Towards Large-Scale Small Object Detection: Survey and Benchmarks" [43], the authors demonstrate that increasing network depth does not necessarily yield performance improvements. Their results highlight the importance of properly designing an effective feature extracting backbone that minimizes information loss.

Furthermore, they claim that a FPN section 2.3.3 is crucial for SOD and must be tailored for use in a detector. Low-level¹ layers are important for SOD but come with a high computational penalty for detection. Higher-level layers are often not usable, as they are optimized not to keep fine-grained features during training. These focus on low spatial resolution, which makes SOD more challenging in those layers. Their results show that the biggest contribution for SOD in their FPN comes from the P_2 ² layer.

3.2.2. Transformer-based small object detection

Another survey compares how transformer-based neural networks perform in the field of SOD [44]. They compare mixed models that also include CNNs, RNNs, as well as FPNs with transformers. The advantages of transformers over other architectures are showcased. These models are good at learning context and attending to important areas, which makes them especially suited for this task.

Transformers handle spatio-temporal analysis differently from CNNs and RNNs. CNNs extend their filters with an additional temporal dimension, producing 3D feature maps. RNNs add memory cells that track temporal dependencies. Transformers, in contrast, split spatio-temporal learning across separate attention blocks. This avoids attending to the entire spatio-temporal domain at once, which would be computationally prohibitive. However, this design typically results in larger models than CNNs or RNNs.

Nonetheless, in their study, they show that transformer and mixed transformer models can outperform state of the art models but require more data than their more conventional networks to train. Outstanding models in their paper are the Group DETR v2 [45] and the Deformable-DETR [46]. While the Group DETRv2 version with 649M parameters has an AP^{small} of 48.4 on the COCO dataset, the Deformable-DETR network (40M parameters) has an AP^{small} of 26.4 with significantly fewer parameters. On video data,³ from the ImageNet VID dataset[47], Deformable-DETR can outperform other

¹low-level layers are closer to the input and have a higher spatial resolution than higher-level layers

² P_1 is the closest FPN layer to the input while increasing numbers are further away from the input

³, only a small number of transformer based variants were tested on this dataset

3. Related Work

variants of DETR but only achieves an AP^{small} of 13.2. Nonetheless, Deformable-DETR introduces a hierarchical approach to transformer-based vision detectors that improves performance for small objects. The survey further shows different transformer-based architectures and how well they perform on the task of SOD. Compared to other architectures, they can perform well in a mixed setup but also demonstrate that smaller networks struggle more with small object detection.

LW-DETR

A more recent approach for transformer based detection takes advantage of the findings in Deformable-DETR and Group DETR v2, named LW-DETR [16], for real time object detection. With their smallest model (12.1M parameters), they demonstrate fast inference of 2.0 ms (on a T4 GPU) while still achieving an mAP of 42.6 and outperforming YOLOv8 [48]. Further proving that transformer based architectures can outperform state of the art models using CNNss/RNNss architectures. However, they do not report how they performed on the AP^{small} metric. Previous work on this, the Master Thesis [18] can shed some light on the performance of the LW-DETR’s AP^{small} . LW-DETR was used as the backbone and detection head. The precision on the F-UAV-D[49] dataset with event data shows an AP^{small} of 26.7.

3.2.3. Event-based SOD

Most of the benchmarks and research for SOD are conducted on regular images and video inputs. Small object detection on event based data is sparse, nonetheless, in [50], a comparison between different models on event-based SOD is presented.

3.2.4. Summary SOD

Effective SOD relies on well-designed feature hierarchies and tailored feature pyramid networks. Combining multiple architectural types can further enhance detection performance. Transformer-based models increasingly outperform CNN- and RNN-based approaches by leveraging attention to capture fine details, though they require more data and larger architectures. Hierarchical designs are essential across CNNs, RNNs, and transformer-based networks.

3.3. Sparse Networks

For the application of detecting drones against a static background, without camera movement, event data is inherently sparse. Making use of this feature and the potential

3. Related Work

performance improvements is the goal of a lot of research [51], [52], [53], [54], [55], and [56].

3.3.1. Event data representations for neural networks

Event data is provided as a data stream (section 2.1). To use this data for different models, it needs to be transformed first into a usable representation. The simplest approach is to create images from the events so that common image based networks can train on that data. In [23], comparisons between different approaches to representing event data and using it for machine learning are presented. To do this, they use the Gromov-Wasserstein Discrepancy (GWD). While demonstrating that their representation of the Event Representation through Gromov-Wasserstein Optimization (ERGO) with 12 channels performs the best, it is also the most involved version to implement. Even their study for the Recurrent Vision Transformer [57] uses the much simpler approach of a voxel grid, where a stacked histogram with 10 time bins and 2 bins for polarity is used. Nonetheless, the input representation can further improve or worsen the networks detection performance and should be considered. Still, most papers using event data utilize the voxel grid representation [58], [57], [59].

3.3.2. Specialized architectures for continuous data

Other approaches do not just use an input strategy but also model their network’s architecture around the idea of a continuous stream of data. One such approach are Spiking Neural Networks (SNNs) that act like neurons, firing only after a certain threshold is met, thus mimicking the function of a brain. This spiking mechanism is not differentiable, and training workarounds are necessary to overcome this hurdle [60], [61], [62]. As of now, there is no research suggesting that SNNs are competitive in the area of object detection[63].

Another approach for making use of the sparsity of event data streams are Graph Neural Networks (GNNs) [51], [52]. GNNs use a graph structure, creating nodes and edges by sub-sampling events and creating a spatio-temporal graph. Aggressive subsampling can lead to losing important information and degrading the performance of the network, while subsampling is necessary for detecting slow moving objects to maintain fast inference rates.

3.4. Filtering

Furthermore, there is the approach of prefiltering the event data [18]. The approach creates patches over the voxel grid event data and only keeps patches with high enough activation. This is done by using a simple threshold. The biggest gain of this approach

3. Related Work

is that the whole network, after the prefiltering, profits from the sparsification. This approach improves inference speed significantly but suffers in detecting small objects.

3.4.1. Scene Adaptive Sparse Transformer (SAST)

Improving the filtering could thus improve the detection models performance on SOD. Instead of using a threshold based approach, a model like SAST [59] is explored in this work to perform the filtering. SAST focuses on extracting feature rich tokens from event data having a capable backbone. The backbone is structured with hierarchical transformers using Long short-term memory (LSTM) (section 2.7) blocks for temporal information. They introduce a scoring mechanism to focus on input features that are important for the detection of objects while also making use of the sparsity of events for lower computational overhead. Their backbone then forwards their selected hierarchical tokens to a FPN and a YOLOX[28] head for detection.

3.4.2. SAST in relation to current event-based detectors

In their work, they introduce an object detector for sparse event data. The results show this architecture outperforms state of the art event based networks like Recurrent Vision Transformer (RVT)[57]. However, SAST does not report the AP^{small} performance. Nevertheless, the model’s architecture and its mAP performance indicate strong potential. Since surveys show that hierarchical designs, hybrid architectures, and attention mechanisms are particularly effective for small object detection, this model is a strong candidate for filtering event data. Furthermore, this model has 18.9M parameters and is rather small for a transformer based object detector. Later in this work, we show the performance of the SAST model on SOD.

3.5. Event–RGB Multimodal Fusion for Object Detection

To further improve object detection, [64] and [65] use a multimodal approach where not only event data but also RGB image data is utilized. Both modalities are fed into the detection pipeline. The fusion of RGB and event data is more involved. Problems of combining the data include synchronizing event and image data, as well as aligning the data from different cameras. Additionally, the limited number of datasets [49], [64], [50], [66] that provide both images and event data makes training and comparison harder.

In [64], they show that they can improve the $AP^{.50:.05:.95}$ by 4.5% using their *Pooling (Encoder)* method for fusing event and RGB data compared to the DETR [34] implementation using event data only.

3. Related Work

	F-UAV-D	NeRDD	1 Mpx		
Models	mAP[%]	mAP [%]	mAP [%]	Time [ms]	Params [M]
YOLOX-RGB [28]	44.4	9.7	-	1.92	9.0
LWDETR-Event [16]	30.6	32.9	-	2.76	8.3
SparseDETR [18]	32.5	-	-	2.6	11.9
SAST [59]	40.7	36.0	48.7	4.98	18.9
RVT-B [57]	55.8	-	47.2	6.1*	18.5

Table 3.1.: mAP of different ANNs on different datasets. Inference measured in ms for a batch of size 1 on NVIDIA Jetson AGX Orin. RVT-B (*) measured on T4 GPU.

	NeRDD	DSEC-Detection	EV-UAV		
Models	mAP [%]	mAP [%]	IoU [%]	Time [ms]	Params [M]
NeRDD (Pool) [64]	39.3	-	-	-	59.2
FAOD [65]	-	42.5	-	14.6	20.3
Ev-SpSegNet [50]	-	-	55.18	39.5	4.0

Table 3.2.: NeRDD and FAOD multimodal approaches with mAP on their respective dataset. The Ev-SpSegNet event based small object detection approach measuring the performance in *IoU*. FAOD inference for a batchsize of 1 on NVIDIA Tesla V100 GPU while Ev-SpSegNet uses a NVIDIA RTX 3090 GPU.

Further, [65] shows how to tackle the misalignment between RGB and event data and how to overcome it with their *Align Module*. Another improvement is their varying training frequency to simulate a real world environment, making their model more robust.

3.6. Summary

In table 3.1, event based and RGB based models are depicted. It shows their corresponding mAP on the datasets used in this work, as well as on the common 1 Mpx dataset. Furthermore, in table 3.2, models with mAP and Intersection over Union (IoU) on their own datasets are displayed. FAOD and NeRDD use a multimodal approach, incorporating both RGB and event data.

Chapter 4

Implementation

This chapter begins by presenting the datasets used in this work, including their main characteristics and visualizations. It then discusses the neural network architectures that were implemented and evaluated.

4.1. Datasets overview

In this section, the two datasets used in the present research are introduced. The first is F-UAV-D [49], and the second is NeRDD [64].

The following subsections first discuss the characteristics of both datasets and then describe the process of integrating them for training and evaluation.

4.1.1. Dataset characteristics

F-UAV-D

The F-UAV-D dataset contains half an hour of images, and event data of drones against a clear sky and forests backgrounds. Not all recordings were captured with a static camera, as the dataset includes both static and moving shots. The distance of the drones to the camera varies, with the dataset being dominated by distant objects.

F-UAV-D offers synchronized event data with RGB images. While the event data is provided as a continuous data stream the RGB images are captured every 33 ms. The provided labels were created by annotating the RGB images, thus the labels are provided at the same frequency as the images. With the provided homography event data, RGB images and labels can be transformed into the same viewpoint.

4. Implementation

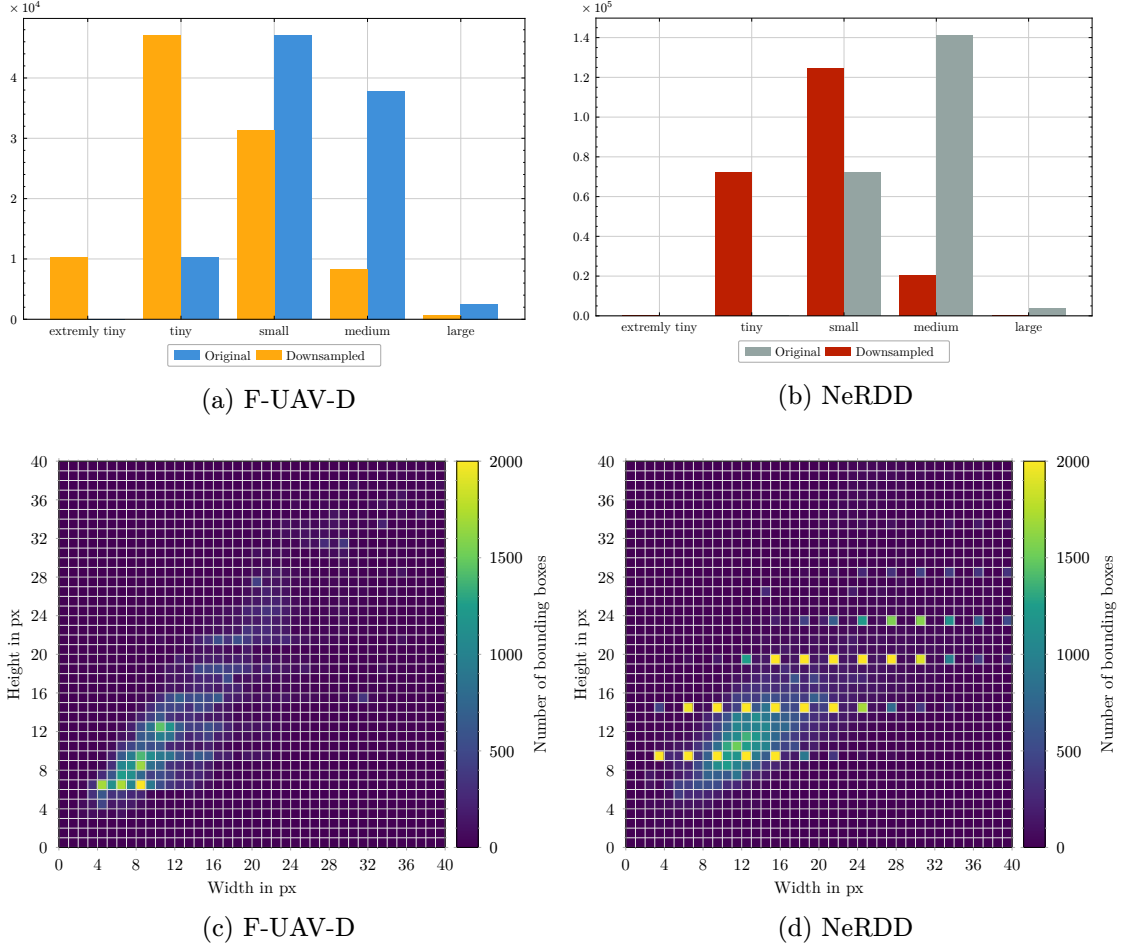
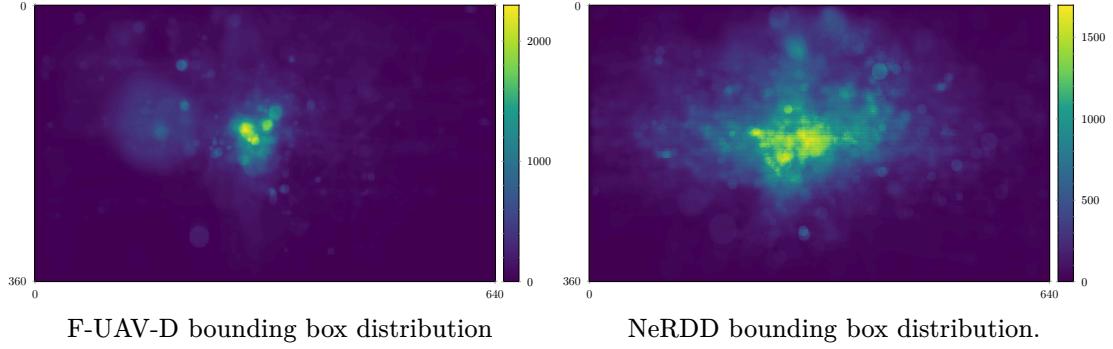


Figure 4.1.: (Top) Distribution of bounding-box size categories for F-UAV-D and NeRDD (original 1280×720 px downsampled to 640×360 px). (Bottom) Height and width distributions (outliers removed, 50 bins).

Figure 4.1(a) presents the number of bounding boxes grouped by size category for the F-UAV-D dataset. Bounding boxes are added into different sized bins defined as extremely tiny = $[0, 8^2)$, tiny = $[8^2, 16^2)$, small = $[16^2, 32^2)$, medium = $[32^2, 96^2)$ and large = $[96^2, \text{inf})$. The categories come from [67] defining small, medium and large bounding boxes with additional subdivisions introduced by [50] to distinguish even smaller objects. In fig. 4.1(c), the height vs width is displayed as a 2D histogram. It shows that most bounding boxes fall into the tiny category, close to the boundary with extremely tiny.

However, under the COCO evaluation protocol, all bounding boxes in the small category and below are grouped together into the small class. This prevents finer-grained distinctions from being reflected in the metrics.

4. Implementation



The figure shows that downsampling increases the number of bounding boxes in the smaller categories. The downsampled F-UAV-D dataset contains many objects from the extremely tiny to small range, making it ideal for studying small object detection.

In section 4.1.1, the spatial distribution of bounding boxes within the event frame is shown. Most bounding boxes are concentrated near the center of the frame, which is not ideal but can be partially mitigated through data augmentation.

NeRDD

The NeRDD dataset provides 3.5 hours of images and event data. All data is recorded from a static tripod without any camera movement. The background varies more compared to the F-UAV-D dataset and shows buildings, backyards, streets and different kinds of vegetation. The images and event frames are already transformed into the same viewpoint.

In fig. 4.1(b), the bounding box size categories of NeRDD is displayed, showing that there is a more prominent distance for drones and thus a lot of similar sized bounding boxes. The prominent size shifts when down sampled and adds most of the bounding boxes to the small category. Overall one can see in ?? that the NeRDD dataset has a smaller size distribution for bounding boxes and those boxes are on average bigger than the ones of F-UAV-D.

The pattern in fig. 4.1(d) indicates that the NeRDD dataset has a lot of static similar sized bounding boxes. That could come from the labeling process but could also indicate that there are a lot of static drone data. This anomaly with the checkerboard pattern is further discussed in section 6.2

In section 4.1.1, the distribution of NeRDD bounding boxes is shown. Compared to the F-UAV-D dataset, the bounding boxes cover a larger portion of the image frame.

4. Implementation

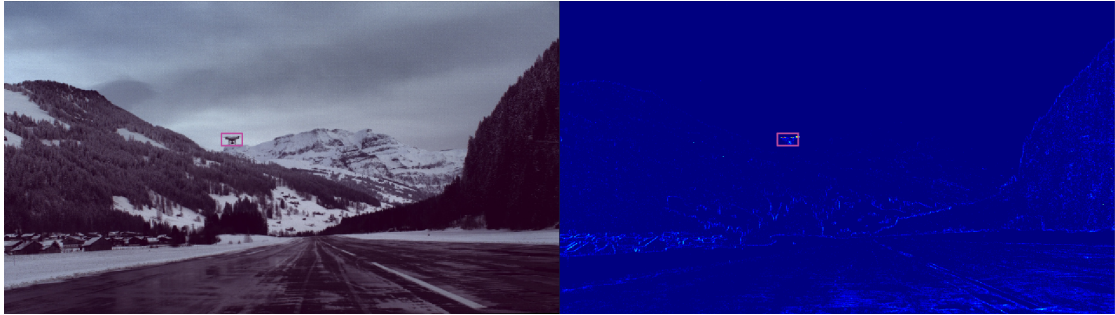


Figure 4.3.: Example of RGB, event data and labels from F-UAV-D



Figure 4.4.: Example of RGB, event data and labels from NeRDD



Figure 4.5.: Example images from the F-UAV-D dataset



Figure 4.6.: Example images from the NeRDD dataset

4. Implementation

4.1.2. Dataset summary

Together, the two datasets provide four hours of drone footage across different environments and object sizes. This variety is valuable for training object detectors for drone detection. Figure 4.3 and fig. 4.4 show sample snapshots from each dataset. In the following section, we describe how the datasets were preprocessed and prepared as input for our models.

4.2. Dataset integration

Event data is often stored in different formats. F-UAV-D stores the continuous event stream in HDF5 [68] files while NeRDD saves it in compressed numpy [69] files. The implemented models require that event data is loaded as a voxel grid (section 2.2) and SAST needs a consecutive sequence of voxel grids. Additionally labels, voxel grids and RGB images need to be synchronized.

To facilitate better integration with our models, the datasets are processed into a more suitable format.

To make the use easier data is processed into one common intermediate format for further usage. The main goal of this format is to be easy understandable and easy to use. Figure 4.7 illustrates the concept of a common format, which enables a single dataloader implementation to handle the data while applying randomization and additional augmentations.

The next section describes the raw layout of the datasets and outlines the steps taken to transform them into the common format.

RAW dataset layouts

F-UAV-D and NeRDD have different raw dataset layouts. F-UAV-D has a folder with RGB images that is time-synchronized with the labels. The labels are stored in a numpy file with attached timestamp. Additionally there is a HDF5 file for the events. Synchronization between labels and event data is done on their respective timestamps.

For NeRDD there is a RGB image folder as well, but the timestamps of the images are inconsistent (see section 6.2). For synchronization of the labels with the image data a capture rate of 30FPS is adopted. Labels and events are synchronized via the timestamp. NeRDD uses a compressed numpy file for the event data and a txt file for the labels.

4. Implementation

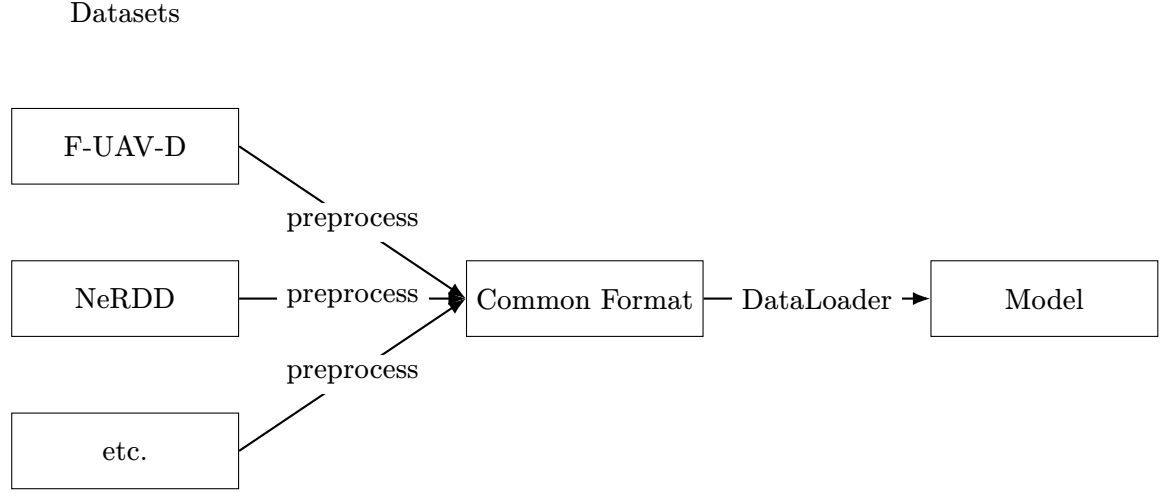


Figure 4.7.: All datasets are formatted into a common format that can be used in a single dataloader thus simplifying the data loading process

Preprocessed layout

The result of preprocessing one of the datasets is displayed in fig. 4.8. Each top level folder (e.g. 2024_01_10_112814_drone_000) contains a full sequence of synchronized events, RGB images and labels i.e. each event_1.npz file contains a StackedHistogram that is time-synchronized with the image_1.npz and label_1.npz. When there is no drone in the frame the label file contains an empty array.

The advantage of this format is that loading synchronized data in the dataloader is as easy as this:

```
event_files = os.listdir(event_path)
label_files = os.listdir(label_path)
rgb_files = os.listdir(rgb_path)

event_files.sort(key=lambda item: (len(item), item))
label_files.sort(key=lambda item: (len(item), item))
rgb_files.sort(key=lambda item: (len(item), item))
```

For training, the event data was preprocessed into voxel grids, each consisting of 10 time bins and 2 polarity bins, with the temporal dimension sampled over 50 ms.

4. Implementation

```
+-- train
|   +-- 2024_01_10_112814_drone_000
|   |   +-- events
|   |   |   +-- event_0.npz
|   |   |   +-- event_1.npz
|   |   |   +-- event_2.npz
|   |   +-- labels
|   |   |   +-- label_0.npz
|   |   |   +-- label_1.npz
|   |   |   +-- label_2.npz
|   |   +-- rgbs
|   |   |   +-- rgbs_0.npz
|   |   |   +-- rgbs_1.npz
|   |   |   +-- rgbs_2.npz
|   +-- 2024_01_10_112814_drone_002
|   |   +-- events
|   |   +-- labels
|   |   +-- rgbs
+-- val
|   +-- 2024_01_10_112814_drone_001
|   |   +-- events
|   |   +-- labels
|   |   +-- rgbs
+   +-- ...
```

Figure 4.8.: Dataset structure on disk after the preprocessing step.

4. Implementation

4.2.1. Data loading

Different models require different types of input for training. While most models rely on single event voxel grids or RGB images with their corresponding labels, the more advanced SAST models make use of recurrent blocks (section 2.7). Training such models requires backpropagation through time (BPTT), and the data must be provided in sequence to train the memory modules.

The dataset layout naturally supports sequential training, as each folder corresponds to a sequence whose max length equals the number of samples it contains. For the dataloader, we adopted the mixed loading implementation from SAST [59]. From the preprocessed data, multiple sequences of length 5 are constructed and provided as input to the networks using SAST blocks. Incomplete sequences at the end of a folder, containing fewer than five steps, are omitted.

4. Implementation

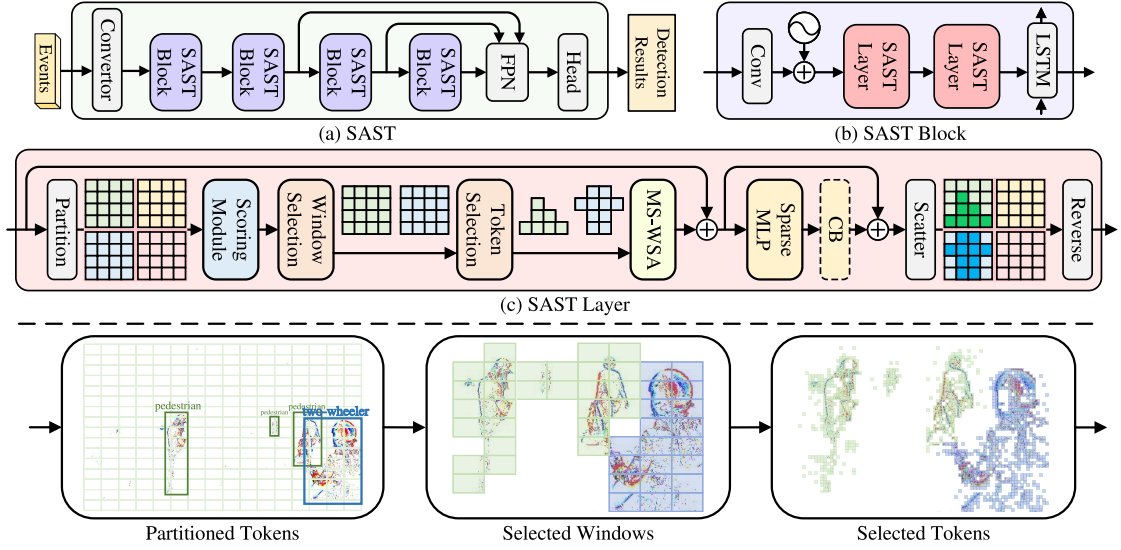


Figure 4.9.: SAST model architecture from [59]

4.3. Neural Network Architectures

To improve the filtering of the previous work see section 2.6 the SAST model was adopted. First the SAST model is introduced and then the different model adaptations are explained.

4.3.1. SAST

In fig. 4.9, the general architecture of SAST is displayed. To utilize the token selection capability of SAST, the model was adapted to be able to train on the F-UAV-D and NeRDD dataset.

SAST has a hierarchical architecture with four SAST-Blocks at different hierarchy levels (0, 1, 2, 3) fig. 4.9. The hierarchy levels allows SAST to extract features from different scales. SAST-Block 1, 2, 3 are fed into a PAFPNs block and further YOLOX detection head.

As displayed in fig. 4.9 SAST-Blocks, are their novel approach for token selection. SAST-Blocks are constructed by two SAST-Layers which in turn each use a scoring and selection module for token selection inside displayed in fig. 4.9c. Each of the SAST layer first partitions the input into tokens and feeds them into a scoring module that is tightly coupled with the selection module.

The job of the scoring and selection module is to extract key tokens while preserving enough context in scenes with fewer events. Furthermore, they propose Masked Sparse Window Self-Attention as a novel solution to the problem that not all selected tokens

4. Implementation

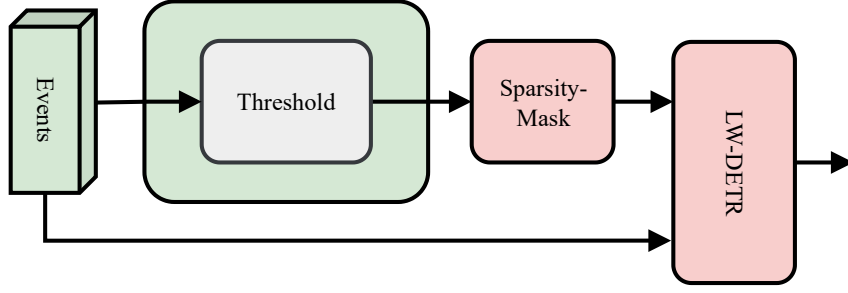


Figure 4.10.: SparseDETR: simple thresholding approach.

have the same size, which is required for self-attention. In this approach, the tokens are first padded to a uniform size so that self-attention can be applied, and are then unpadded afterward to prevent information leakage from the padding.

4.3.2. Modifications to the original SAST

In our approach, the SAST model was modified to accommodate data with unlabeled frames, addressing a limitation of the original architecture, which required each sequence to contain at least one annotated frame. Additionally the filtering of small bounding boxes in the evaluation process was removed. This is to accommodate the F-UAV-D (section 4.1.1) dataset with most bounding boxes being in the small category. Otherwise the evaluation of the SAST implementation would filter those boxes.

4.3.3. SAST+LW-DETR

The modified SAST module is used to replace the simple filtering done in [18]. For the previous work a simple approach called SparseDETR was implemented. An overview of this architecture is displayed in fig. 4.10 and discussed more in section 2.6. In this approach a simple threshold-based sparsity mask is created. This sparsity mask, together with the original event data, is provided to a modified LW-DETR implementation to process only the patches that exceed the event threshold. As the LW-DETR is used for detection, the model without any filtering sets an upper limit on how good models with filtering can perform. SparseDETR, and shown in this work SAST+LW-DETR, filter the input to LW-DETR and only remove information thus not being able to outperform the LW-DETR model itself. Faster learning and convergence on the other hand can improve.

In fig. 4.11, the updated approach is shown. Instead of the threshold-based algorithm, the SAST network is used to create the sparsity mask. The lowest pyramid layer outputted

4. Implementation

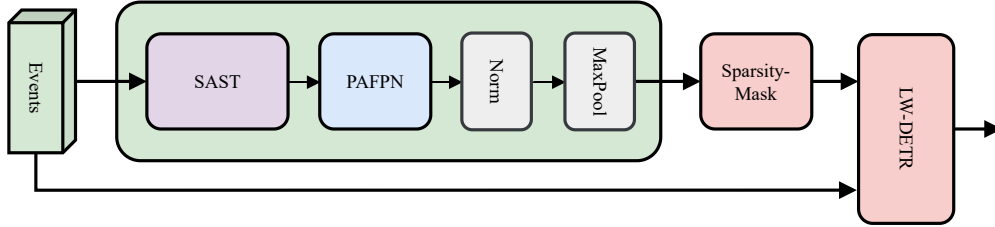


Figure 4.11.: SAST+LW-DETR architecture overview.

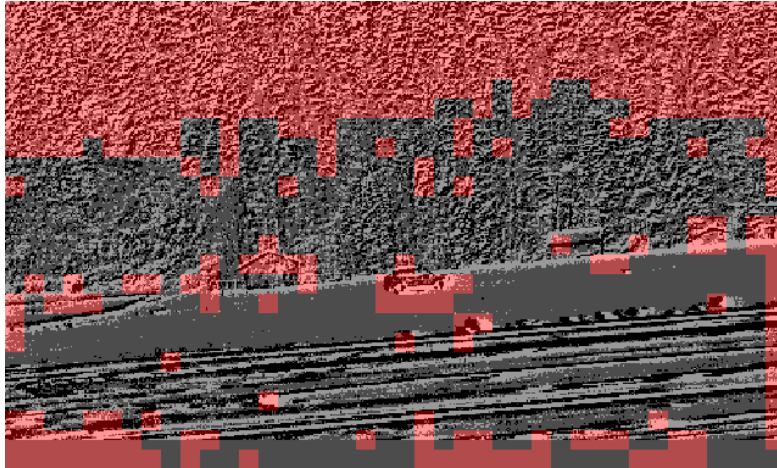


Figure 4.12.: Sparsity mask created from SAST.

4. Implementation

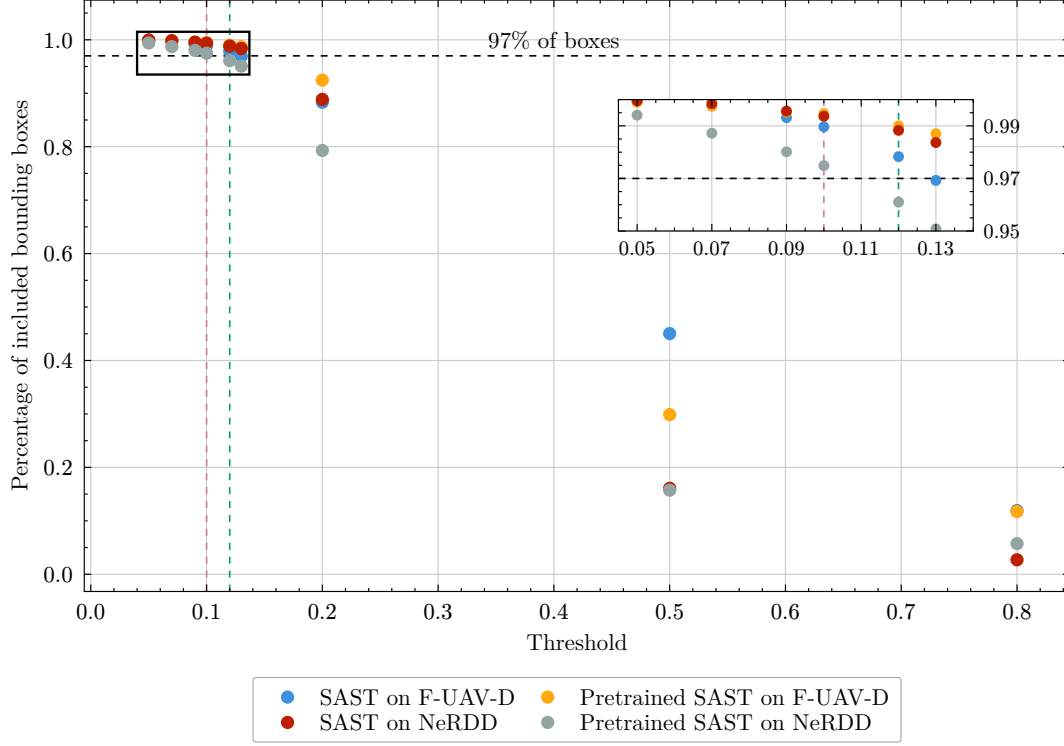


Figure 4.13.: Different thresholds retain different percentage of bounding boxes. The green and purple vertical lines indicate the chosen thresholds which retain more than 97 percent of the bounding boxes. They represent the selected value for the SAST checkpoint and pretrained SAST checkpoint respectively

by the PAFPNs is used in combination with a norm and max pool block. In a first step, the norm block normalizes the output of the PAFPNs to be between 0 and 1. Then, the max pool ensures that each patch in the mask is 16×16 px and aligns with the event data patches. A threshold converts this mask to a binary mask as the final step to use it as the sparsity mask for filtering in the LW-DETR block. In fig. 4.12, a created sparsity mask by this process is displayed. The threshold for selecting event patches was chosen by analyzing, that a generated mask by the SAST model retains at least 97% of all bounding boxes. The fig. 4.13 shows that a threshold of 0.1 (purple vertical line) and 0.12 (green vertical line), for the SAST checkpoint and for a pretrained SAST model on F-UAV-D respectively, retains the most boxes while being semi sparse.

4.3.4. SAST+RGB

Furthermore we also implemented a model architecture that includes both modalities, the event data, as well as the RGB images. In this setup, the goal is to further improve the detection performance at the cost of inference speed.

In fig. 4.14, the overall architecture is displayed. Instead of creating a sparsity mask from the SAST module, the hierarchical output is combined with a YOLOX backbone whose input are the RGB images. Detection is done by a YOLOX detection head on the combined hierarchical data.

To extract the same amount of semantic information from the RGB data as SAST does from the event data, YOLOX is employed as the backbone for the RGB stream. After feature extraction, the outputs of both networks are aligned at the PAFPN stage and combined, ensuring that information from the two modalities is fused at a comparable semantic level. This design allows each modality to leverage a strong, specialized architecture while enabling effective multimodal integration.

Alternative approaches using smaller RGB backbones led to degraded performance, as the high information density of the RGB stream tended to overshadow the event data, while the reduced capacity of the backbone limited its ability to extract sufficient features. In essence, the network depth was insufficient to balance the two modalities effectively.

Another advantage of this implementation is that it facilitates a direct comparison of the impact of RGB image detection on overall performance, since both modules, SAST and YOLOX, can also be run independently and evaluated separately.

4. Implementation

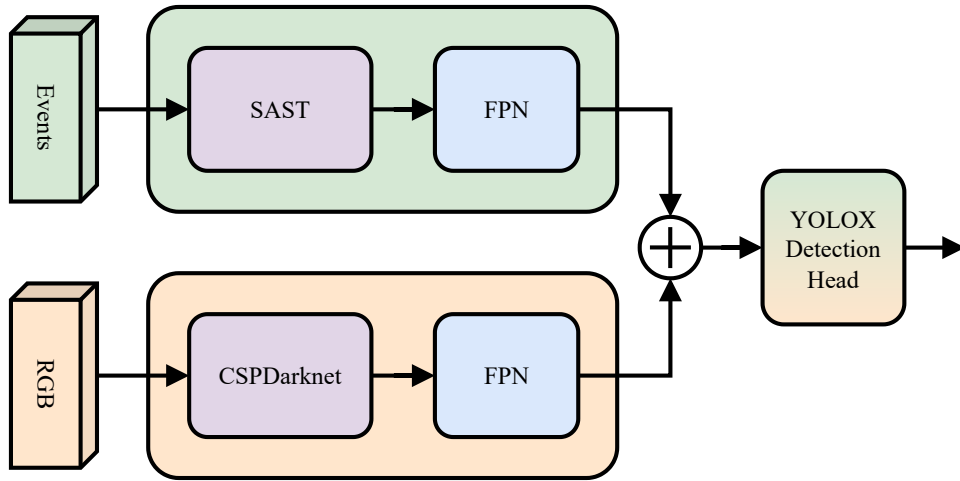


Figure 4.14.: SAST+RGB architecture overview

Chapter 5

Results

This chapter presents the results of the different models. First, the test setup is described, along with the methodology used, and then our findings are presented. As small object detection and performance are the main goals of this work, metrics and figures will primarily focus on these aspects. Nonetheless, more information is shown when there are interesting findings in other areas.

5.1. Models

For the experiments, different model architectures were used. For a better overview, table 5.1 shows what data each model uses, as well as whether the model contains transformer or recurrent blocks. The filtering column emphasizes that the SAST part in SAST+LWDETR and SAST-Pretrained+LWDETR is only used for filtering. The filtering is explained further in section 4.3.3. The YOLOX-* and LWDETR-* are the base models serving as a reference. They are further utilized in combination with SAST. SAST+RGB is the model that works on both modalities, event and RGB data, which is explained further in section 4.3.4. All models, except for LWDETR, were trained with pretrained weights from the respective repositories.

SparseDETR in the middle of table 5.2 is from [18] and showcases the sparse filtering approach (section 2.6)

For LWDETR the models performed better without pretrained weights. This could be due to the changed input size but was not further investigated.

5. Results

Model	RGB	Event	Transformer	Recurrent	Filtering
YOLOX-RGB	✓	✗	✗	✗	✗
YOLOX-Event	✗	✓	✗	✗	✗
LWDETR-RGB	✓	✗	✓	✗	✗
LWDETR-Event	✗	✓	✓	✗	✗
SparseDETR	✗	✓	✓	✗	✓
SAST	✗	✓	✓	✓	✗
SAST+RGB	✓	✓	✓	✓	✗
SAST+LWDETR	✗	✓	✓	✓	✓
SAST-Pretrained+LWDETR	✗	✓	✓	✓	✓

Table 5.1.: Overview of different tested models and their architectures

5.2. Experimental Setup

Different model architectures are compared for their accuracy on COCO’s AP^{small} and mAP, as well as for their inference speed. For this, all models were trained either on the F-UAV-D, NeRDD, or a mixture of those two datasets. Furthermore, checkpoints were used for different models, and it is highlighted in the model or description which part of the model used pretrained weights.

Training and evaluation were conducted on the NeRDD and F-UAV-D datasets. Both datasets have an 80:20 split for training and evaluation, respectively. Furthermore, data augmentation was applied with 50% chance of horizontal flips and 80% probability of zoom. Zoom-in was configured to be twice as likely as zoom-out. The zoom factor for zoom-in was chosen to be 1.5 and 1.2 for zoom-out.

Each model on the datasets was run for 20 epochs, and the best mAP with its corresponding AP^{small} is displayed.

5.3. Precision experiments

The first experiment table 5.2 compares different model performances on the F-UAV-D dataset. In table 5.3, the second experiment on the NeRDD dataset is displayed. To compare the two datasets, F-UAV-D and NeRDD, only 25% of the NeRDD training and evaluation set was used. This makes them comparable in size and shows how each model performs on each dataset. To see if more data improves the models performance in the third experiment table 5.4, all models were trained on 100% of the F-UAV-D dataset, with an additional 20% of data from the NeRDD dataset.

5. Results

F-UAV-D			
Model	Pretrained weights	AP ^{small}	mAP
YOLOX-RGB	YOLOX-Tiny	44.0	44.4
YOLOX-Event	YOLOX-Tiny	38.8	40.5
LWDETR-RGB	$\times$	13.9	13.3
LWDETR-Event	$\times$	28.2	30.6
SparseDETR	LWDETR-Tiny*	20.4*	32.5*
SAST	1 Mpx	39.5	40.7
SAST+RGB	1 Mpx + YOLOX-Tiny	48.6	50.1
SAST+LWDETR	1 Mpx + $\times$	26.1	28.5
SAST-Pretrained+LWDETR	F-UAV-D + $\times$	26.7	28.6

Table 5.2.: Performance of different networks on the F-UAV-D dataset. The top part shows base model performance while the bottom part shows SAST and our SAST variants. SparseDETR is the previous filtering work and values with * are reported from [18].

NeRDD			
Model	Pretrained weights	AP ^{small}	mAP
YOLOX-RGB	YOLOX-Tiny	9.9	9.7
YOLOX-Event	YOLOX-Tiny	31.9	31.9
LWDETR-RGB	$\times$	0.0	0.0
LWDETR-Event	$\times$	32.6	32.9
SAST	1 Mpx	35.7	36.0
SAST+RGB	1 Mpx + YOLOX-Tiny	36.7	37.3
SAST+LWDETR	1 Mpx + $\times$	29.0	29.3
SAST-Pretrained+LWDETR	F-UAV-D + $\times$	28.6	28.9

Table 5.3.: Performance of different networks on the NeRDD dataset. The top part shows base model performance while the bottom part shows SAST and our SAST variants.

5. Results

Mixed			
Model	Pretrained weights	AP ^{small}	mAP
YOLOX-RGB	YOLOX-Tiny	33.8	34.5
YOLOX-Event	YOLOX-Tiny	35.2	36.5
LWDETR-RGB	$\times$	12.7	12.4
LWDETR-Event	$\times$	30.0	31.8
SAST	1 Mpx	40.9	42.5
SAST+RGB	1 Mpx + YOLOX-Tiny	43.7	45.0
SAST+LWDETR	1 Mpx + $\times$	26.5	28.1
SAST-Pretrained+LWDETR	F-UAV-D + $\times$	27.6	29.0

Table 5.4.: Performance of different networks on 100% of F-UAV-D and 20% of NeRDD. The top part shows base model performance while the bottom part shows SAST and our SAST variants.

5.4. Inference experiment

All performance inference tests were conducted on a NVIDIA GeForce RTX 4090 and a NVIDIA Jetson AGX Orin.

5.4.1. NVIDIA GeForce RTX 4090 pytorch

The tests on the NVIDIA GeForce RTX 4090 were conducted by running an unoptimized model on the GPU with a single input image and event frame. The event and image frame are of size 640×384 px. For measuring the inference, each model underwent 10 warm up runs followed by 2000 "hot" runs. The average of those 2000 runs is displayed in table 5.5. For SAST+LWDETR, different sparsity levels increase performance, as discussed in section 6.4.3. The table 5.5 shows the worst case scenario of no sparsity for SAST+LWDETR.

Performance tests with PyTorch were highly sensitive to server load, so all measurements were taken when CPU and GPU usage was minimal. Unlike TensorRT, PyTorch supports varying transformer input sizes, important for the filtering approaches (SparseDETR, SAST+LWDETR). Moreover, PyTorch benchmarks are easier to reproduce, providing a more accessible point of reference for others.

5.4.2. NVIDIA Jetson AGX Orin TensorRT

Inference tests on the NVIDIA Jetson AGX Orin utilize the TensorRT [70] optimization. First, the models were converted to an ONNX [71] model. This model is optimized

5. Results

with the ONNX optimizer and is further used in the TensorRT engine for inference tests. The command shown in fig. 5.2 was used to run inference tests. Converting the models to ONNX was straightforward, but further optimization and conversion to TensorRT required modifications to the SAST models, as well as to SparseDETR and SAST+LWDETR. The sparse LWDETR models, in particular, had to be adapted since TensorRT does not support variable tensor sizes within transformer blocks. To address this, a static sparsity mask was utilized. In addition, Scene Adaptive Sparse Transformer (SAST) required adjustments because certain PyTorch operations, such as the *tensor_split* instruction, are not supported in TensorRT. These limitations make direct comparisons with TensorRT more difficult. Once the TensorRT engine file is generated, running tests on the NVIDIA Jetson Orin becomes straightforward.

5.4.3. Parameter Counts, FLOPs, and Resolution Effects

GFLOPs and parameter count were measured using the THOP Python library [72] from ultralytics.

The SparseDETR inference values, except for the RTX 4090, are from [18]. It is important to note that SparseDETR uses an input tensor of 640×640 px, while all other performance tests use 640×384 px. The decrease in input size improves the performance of all the models and is discussed in section 6.4.3. For all models with filtering, a range is provided for the inference time. While the lower number represents the best case of 1% patches, the higher number represents the worst case where the whole frame has to be computed.

5. Results

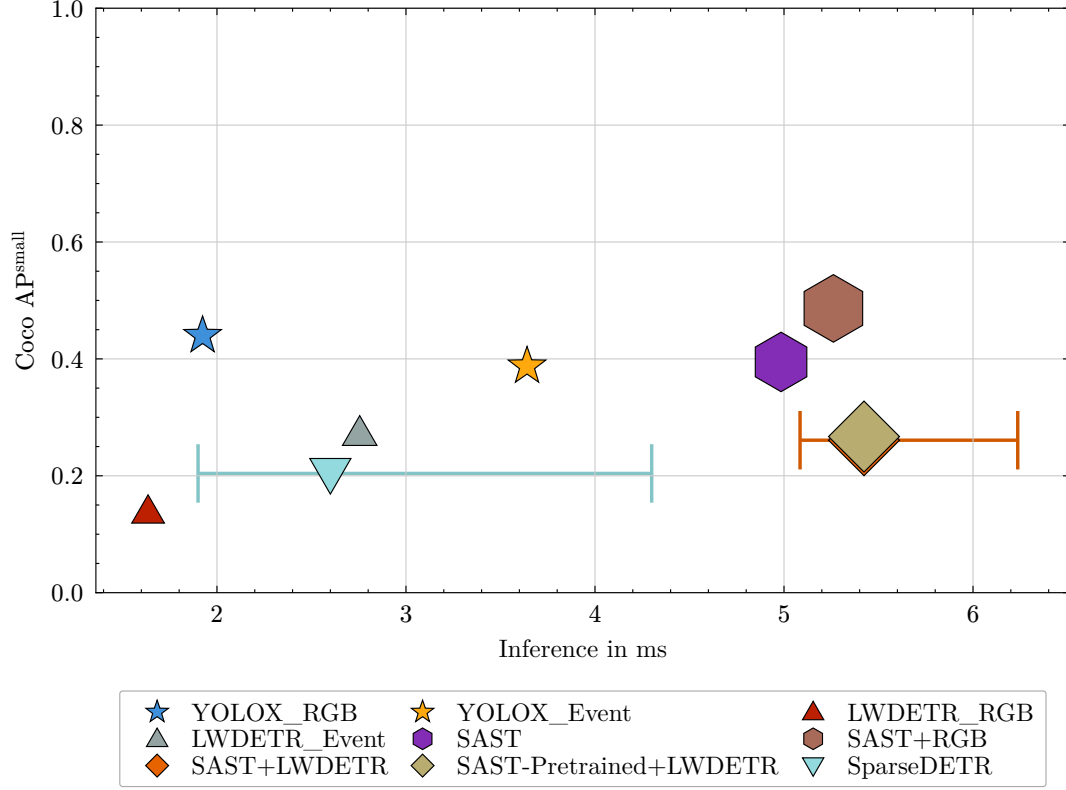


Figure 5.1.: Models trained and evaluated on F-UAV-D with the corresponding AP^{small} . The size of the shapes represents the model size. Inference is on the NVIDIA Jetson AGX Orin in ms. SAST+LWDETR is hidden under SAST-Pretrained+LWDETR. For SparseDETR and SAST+LWDETR models the point shows the median inference time while the whiskers show the best and worst case inference. Best case is represented by 1% patches while worst case is represented by 100% patches.

```
trtexec --useCudaGraph --useSpinWait --warmUp=500 --avgRuns=1000 --
fp16 --loadEngine=$optimized_onnx_model
```

5.2 Command used for inference tests

5. Results

Model	Params [M]	Inference		
		GFLOPs	RTX 4090 [ms]	AGX Orin [ms]
YOLOX-RGB	9.0	16.2	2.61	1.92
YOLOX-Event	9.0	18.6	X	3.64
LWDETR-RGB	7.5	8.0	3.65	1.63
LWDETR-Event	8.3	9.6	3.89	2.76
SparseDETR	11.9*	7.4*	4.5 - 7.0	1.9 - 4.3*
SAST	18.9	42.0	6.55	4.98
SAST+RGB	31.3	52.3	7.92	5.26
SAST+LWDETR	30.7	43.9	9.72 - 10.18	5.09 - 6.24

Table 5.5.: Inference time comparison across multiple architectures. Performance on the NVIDIA GeForce RTX 4090 was measured using pytorch and CudaTimer without optimizations, while results on the NVIDIA Jetson AGX Orin were obtained using TensorRT-optimized models. SparseDETR values with * is data reported from [18]

Discussion

After presenting the results, this chapter provides a thorough analysis of them. The findings are examined in relation to the research objectives, compared with existing literature, and evaluated with respect to their limitations and relevance.

First, the dataset sections present model performance and explain the behavior of the baseline models. This is followed by two sections that examine our contributions in more detail, with one addressing filtering of sparse event data (section 6.4) and the other the multimodal approach (section 6.5).

6.1. F-UAV-D

On the F-UAV-D datasets the base YOLOX models (upper part of table 5.2) have a AP^{small} of more than 38%, which can be considered high. The F-UAV-D dataset contains images of drones that are easier to detect, as many shots capture them against a clear sky.

LW-DETR on the other hand, a similar sized model to YOLOX, performs worse with an AP^{small} of 28.2% on the event data. Transformer-based models typically require large amounts of training data and many iterations [31]. Nevertheless, LWDETR-Event converges after 20 epochs. This sets a baseline for comparing other models using LWDETR. Moreover, the LWDETR-Event model sets an upper limit to the performance of our SAST+LWDETR models (see subsection 4.3.3).

The base SAST network, while performing well on the dataset, is outperformed by the much simpler and smaller YOLOX. Nonetheless, table 5.2 shows that using both modalities, RGB and event data, improves the detection performance. The SAST+RGB configuration shows an increase of 4.6% in detection rate on small objects compared to YOLOX-RGB and an increase of 9.1% compared to the base SAST model.

6.2. NeRDD

Table 5.3 shows that transformer-based models perform better on the NeRDD data set compared to the results achieved on F-UAV-D. This can be attributed to the characteristics of the NeRDD dataset, which contains a higher proportion of larger objects. Transformer-based models, owing to their patch-based representation, are known to struggle particularly with the detection of smaller objects [46]. Furthermore, the NeRDD dataset contains fewer small objects, and those that are present are comparatively larger (see fig. 4.1). As a result, transformer-based models achieve better performance on this dataset.

SAST has a higher accuracy than YOLOX-Event showing that it is more robust on more complex event scenes. The results also highlight that transformer-based models improve on the NeRDD dataset, whereas the performance of all other architectures degrades compared to F-UAV-D. Especially, models that use RGB data experience a great decay in performance. The YOLOX model barely reaches a AP^{small} of 9.9%. This can be explained not only by the more complex backgrounds and scenes, but also by inaccuracies in the ground-truth bounding boxes of the RGB image data. Both factors contribute to the reduced detection performance.

The observed misalignment likely originates from the dataset’s use of a single homography for calibration. Such an approach ensures alignment only at one reference depth, causing bounding boxes derived from the RGB images to match the event data at that depth but to become inaccurate at other distances.

With the provided data it is not possible to better align the bounding boxes. In fig. 6.3, such a misalignment is shown. The bounding box is misaligned on the RGB image while the box is good on the event data.

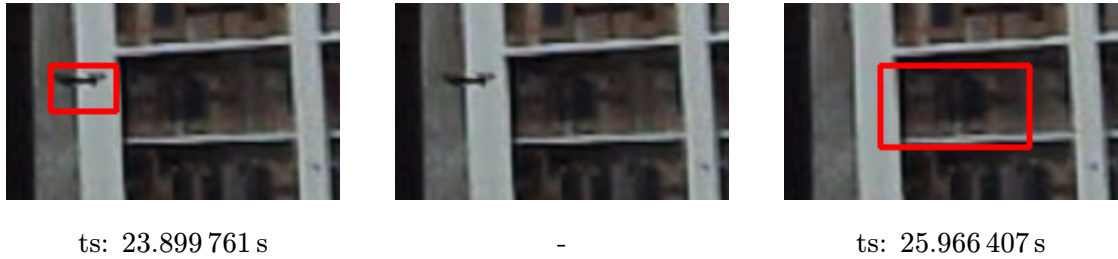


Figure 6.1.: Cropped frame from the NeRDD dataset. On the left a timestamp at 23.899 761 s showing the last bounding box for ≈ 2 s until timestamp 25.966 407 s where the next bounding box "appears". The image in the middle shows the next frame after 23.899 761 s.

Furthermore, the dataset is labeled on the event data which leads to missing bounding boxes when the drone is relatively static. In fig. 6.1 is shown that after timestamp 23.899 761 s there is no bounding box until timestamp 25.966 407 s. The same figure

6. Discussion

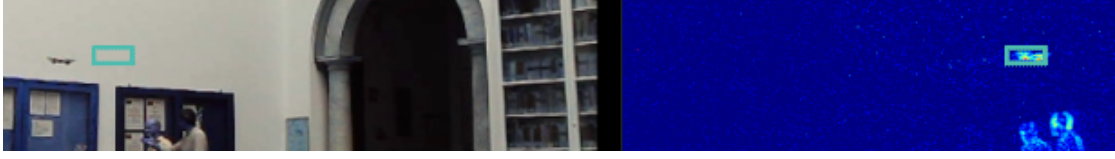


Figure 6.2.: Misaligned bounding box between RGB and event data. This occurs as there is a timestamp issue in the image data where the timestamp of the image increases by multiple seconds but the scene does not jump forward.



Figure 6.3.: Bounding box after synchronizing with 30FPS.

shows another phenomena with the dataset that in the most right image the bounding box stays the same until frame 27.999 720 s (for 2 seconds). This bounding box is static and way bigger than the encompassed drone. In the frame immediately after 27.999 720 s, the bounding box shrinks abruptly as soon as the drone’s movement generates more event activity, suggesting some labeling error/artifact. In fig. 4.1(d), a checkerboard pattern can be seen. We suspect this comes from those static bounding boxes. There are many more examples like this but shown here is data from "NeRDD/DATA/Archive_18/5".

The RGB data is timestamped using the values encoded in the file names, but these timestamps are not consistent across the dataset. Synchronizing the image data with the event and label data based on the RGB timestamps causes problems. In the raw sequence data, some image timestamps jump forward by several seconds even though the scene itself remains continuous.

Thus all bounding boxes after such a time skip are misaligned as they lag behind when doing synchronization on the timestamps. This phenomena is visualized in fig. 6.2. Right before this image a time skip of approximately 2 s occurred rendering all following bounding boxes useless. This lead us to synchronize data by using a static frame-rate of 30 and not relying on the timestamps of the images. In fig. 6.3, the new better bounding box is displayed, which is still not optimal but could be attributed to the misalignment of RGB and event data.

6.3. Mixed

In table 5.4, the results of training and evaluating models on both F-UAV-D and NeRDD is displayed. Overall transformer based models trained on event data (LWDETR-Event, SAST+LWDETR, SAST-Pretrained+LWDETR) lose a bit of accuracy compared to

6. Discussion

only training on NeRDD while gaining a bit compared to training only on F-UAV-D. This small difference stems from NeRDD introducing bigger objects to increase the performance of transformers while F-UAV-D introducing smaller objects decreasing the evaluation performance.

All models using RGB data (YOLOX-RGB, SAST+RGB) have a worse accuracy compared to the F-UAV-D only trained models due to the shortcomings of the RGB labels of NeRDD. The only model that performed better on the combined dataset is SAST. SAST uses only event data and can profit from the good event data of NeRDD even though there are sometimes missing labels.

Overall, this test remains inconclusive, as issues with the NeRDD dataset skew the results. Nevertheless, the SAST+RGB model shows considerable potential, and it is reasonable to expect that a well-labeled dataset would lead to substantial performance improvements.

6.4. Filtering models

SAST+LWDETR is close to the LWDETR-Event base-model on all three tested datasets. The difference to the base-model’s accuracy on all datasets is $\approx 3\%$. This is an increase in accuracy of 10% for SOD compared to the filtering model SparseDETR of [18]. The SAST-Pretrained+LWDETR model, using a pretrained SAST model on F-UAV-D, is further increasing the accuracy by 1% on the Mixed and F-UAV-D dataset. Overall the SAST+LWDETR model is more robust and shows a better performance on the NeRDD dataset.

6.4.1. Effect of Pretraining on SAST+LWDETR

In fig. 6.4, masks for the two different models, SAST+LWDETR on the left and SAST-Pretrained+LWDETR on the right, are displayed. The sparsity mask for both models is 24×40 and thus a max of 960 patches can be selected. Each patch has a size of 16×16 px.

Both models use the threshold that retains more than 97% of patches. Figure 6.4 illustrates a best-case scenario where the F-UAV-D pretrained model primarily selects patches that contain drones. In contrast, fig. 6.5, which depicts a similar scene, shows that the SAST-Pretrained+LWDETR model selects significantly more patches than SAST+LWDETR, including many that do not contain relevant information. Overall, the SAST+LWDETR model without pretraining on F-UAV-D appears more robust to noise and generates more concise masks.

6. Discussion

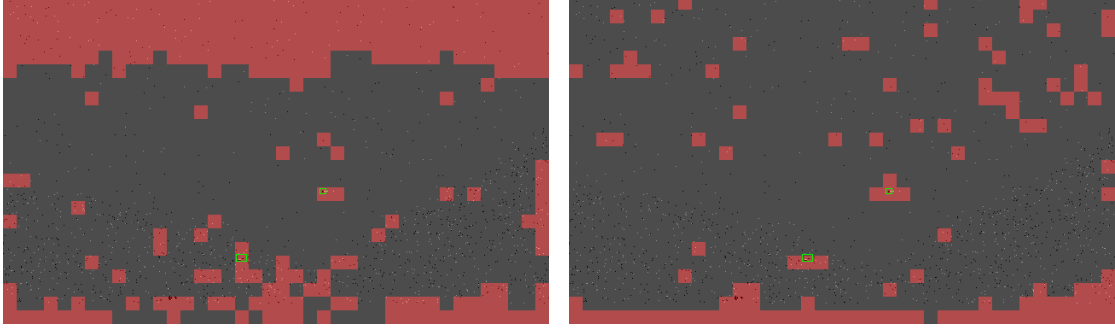


Figure 6.4.: Best case of masks created by SAST checkpoint on the left and on the right by pretrained SAST. Input is from F-UAV-D.



Figure 6.5.: On the left mask created by SAST checkpoint. Right mask created by pretrained SAST on F-UAV-D.

6.4.2. Patch retention and its impact on accuracy

To compare how aggressive the filtering is done table 6.1 shows the average retained patch count over the specified datasets. In this table SparseDETR is included as well, but it has to be stated that the input to SparseDETR is 640×640 px and thus the sparsity mask size is 40×40 . Although the network input is 640×640 px, the source data originates from 640×360 px and is upsampled to match this size. This explains the small average retain rate of 15 %.

Further, table 6.1 shows that SAST-Pretrained+LWDETR retains the most patches and thus has the most information to accurately detect drones. Nonetheless the table shows that an average of approximately 43 % patches is enough to get close to the upper bound accuracy of LWDETR using the more robust SAST+LWDETR model. Additionally, an accuracy increase of more than 5 % compared to SparseDETR on SOD could be achieved.

6.4.3. Trade-offs between filtering, model size, and inference speed

This accuracy increase comes at the cost of inference speed and model size. While the SparseDETR model has a parameter count of 11.9 M the SAST+LWDETR is nearly

6. Discussion

Models	Datasets		Metrics	
	F-UAV-D	NeRDD	AP ^{small}	Median Inference [ms]
SparseDETR	243	268	20.4*	2.6
SAST+LWDETR	414	472	26.1	5.42
SAST-Pretrained+LWDETR	527	490	26.7	5.42

Table 6.1.: Average number of patches retained.

thrice the size with 30.7 M parameters. Moreover, 18.9 M of these parameters are dedicated to filtering and generating the sparsity mask, discarding the feature information extracted by the SAST component.

The inference for the SparseDETR is the lowest with a median of 2.6 ms. It has a bigger input image but the true input to the LWDETR backbone on average is only 243 patches. This size difference leads to a bigger variance. In fig. 5.1, the variance for the SparseDETR is the biggest. Depending on the sparsity level both architectures, SparseDETR and SAST+LWDETR, can show improvements in speed, while the possible speed improvement for SAST+LWDETR is smaller due to the smaller input size. The results show that decreasing the input size is better than filtering.

LWDETR trained with an input size of 640×384 px achieves higher accuracy and a stable inference time independent of sparsity. Moreover, its inference is only 0.16 ms slower than the median SparseDETR runtime.

These results indicate that filtering can be valuable for larger inputs, since transformer complexity scales quadratically with input size. More advanced filtering mechanisms can improve accuracy, but they offer diminishing returns at smaller resolutions. Even at higher resolutions, the added cost of using a large filtering model must be weighed carefully against the potential performance gains.

6.5. Multimodal SAST+RGB

While the approach of filtering has overall more downsides the approach of using both RGB and event data is promising. Not only is the model the best in accuracy on all datasets, the inference time is only 5.26 ms.

Compared to SparseDETR that is an AP^{small} increase of 28.2 % while only taking twice the time for inference. It is further robust on all datasets and more complex scenes. In fig. 6.6, bounding boxes created from YOLOX-RGB model are displayed. The model is detecting bounding boxes in the forest area even though there are no drones in this area. SAST+RGB on the other hand can handle situations like that. It detects all drones perfectly in a similar scene fig. 6.7. Furthermore, SAST+RGB also successfully detects drones when both the event activity is high and the RGB background is more

6. Discussion



Figure 6.6.: Bounding boxes of YOLOX-RGB on a complex background.

Model	Parameters [M]	mAP	$AP^{IoU=.75}$
NeRDD Pooling (Encoder)	59.2	39.3	27.2
SAST+RGB (ours)	31.3	42.2	42.2

Table 6.2.: Comparison of our SAST+RGB model against the best model of [64] on the NeRDD dataset. SAST+RGB was trained on the full dataset for 10 Epochs for this comparison.

complex as shown in fig. 6.8. Additionally, SAST+RGB outperforms the NeRDD Pooling (Encoder) model from [64] on its own dataset while requiring only half the parameters. The mAP and $AP^{IoU=.75}$ results for both NeRDD Pooling (Encoder) and SAST+RGB are summarized in table 6.2. We showed that the SAST+RGB model is a good small object detector with an average performance of AP_{small} of 43%. This demonstrates that incorporating RGB data alongside event data can indeed improve detection accuracy, particularly in complex scenes where RGB alone struggles.

Furthermore, our results show that the proposed multimodal neural network consistently outperforms single-modality baselines (YOLOX, LWDETR, and SAST) and remains competitive with other multimodal approaches (NeRDD), while maintaining a relatively compact size of 31.3 million parameters.

6. Discussion

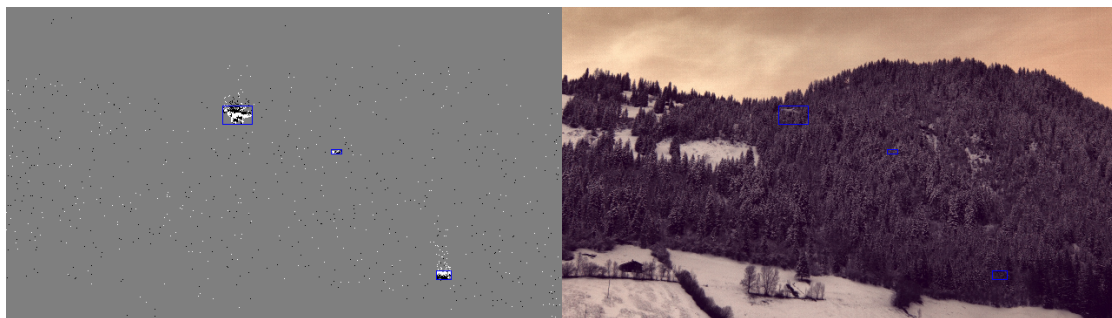


Figure 6.7.: Bounding boxes of SAST+RGB on a complex background with low event activity.



Figure 6.8.: Bounding boxes of SAST+RGB on a complex background and high event activity.

Conclusion and Future Work

This chapter summarizes our findings and gives an overview of where we see potential for further work.

7.1. Conclusion

7.1.1. Filtering of event data

Filtering input to make use of sparse event data can be useful for larger input data but gives diminishing returns when applied to smaller inputs.

With inputs of size 640×384 px, the backbone achieves a maximum inference speedup of 1.2 ms. For an input size of 640×640 px the maximum improvement is 2.4 ms. To improve, filtering face a strict constraint in that accuracy must increase while inference stays under 2.4 ms, a demanding requirement for neural network-based methods.

Our implementation of SAST+LWDETR achieved a 5.7% increase in accuracy, but this came at the cost of an additional 3 ms in inference time. Moreover, even the LWDETR base model outperforms both filtering approaches in small object detection, while remaining nearly as fast as SparseDETR simply by reducing the input resolution from 640×640 px to 640×384 px.

For small object detection, selecting an appropriate model architecture and input size is more critical for balancing speed and accuracy than relying on filtering for speed gains. Filtering can still serve as an optimization step, but it must be applied carefully, ensuring that any speedup justifies the associated slowdown and potential loss in accuracy.

7. Conclusion and Future Work

7.1.2. Multimodal approach

Overall, the multimodal approach demonstrated clear success. SAST+RGB achieves superior performance across datasets and even outperforms the NeRDD baseline on its own benchmark. Across all datasets, the AP^{small} stays above 36.7%.

The combination of a recurrent transformer with CNN-based feature extraction on RGB data provides a strong backbone for SOD. The fusion of modalities allows the model to handle the complexity of RGB scenes by leveraging the often simpler but temporally rich event data, resulting in stable performance under both challenging backgrounds and high event activity. Despite this multimodal design, the architecture remains efficient with 31.3 million parameters and adds only 2.3 ms to the inference time compared to the fastest SparseDETR model, while delivering 1.5× higher accuracy on SOD.

7.2. Future work

Building on the results presented here, the next section introduces potential directions for future work and further development.

7.2.1. End to end pipeline

The current work assumes perfectly synchronized data and does not account for potential misalignments between event streams and RGB images. To fully demonstrate the robustness of the multimodal approach it should be validated under real-world conditions where synchronization issues are present. This may also require adapted training strategies [65]. In addition, the inference tests reported here measure only the raw network runtime, while the overhead of preparing live data into the required input format was not considered. Future investigations should therefore evaluate the full end-to-end pipeline including data acquisition and preprocessing.

7.2.2. Filtering

Filtering becomes increasingly relevant when the input resolution to neural networks grows, since the computational cost of transformer backbones scales quadratically with the number of patches. One possible strategy is to design lightweight backbones that process only a limited number of selected patches, where filtering acts as a front-end to constrain the input. Current event cameras typically provide relatively low resolutions, with 1280×720 px being the standard, so the benefits of filtering remain limited. However, future generations of higher-resolution event sensors could make filtering more impactful, particularly for small object detection, where additional spatial detail is valuable.

7. Conclusion and Future Work

7.2.3. Truncated Backpropagation Through Time (TBPTT)

An immediate extension would be to implement truncated backpropagation through time (TBPTT) for the multimodal SAST+RGB model, which could further improve performance. In SAST [59], it was shown that mixing TBPTT with standard backpropagation through time (BPTT) (currently used) leads to performance gains. Applying a similar strategy in the multimodal setting would likely enhance the effectiveness of SAST+RGB by allowing the model to capture longer temporal dependencies.

7.2.4. Inference improvements

The SAST+RGB model combines recurrent transformers with convolutional layers, which makes it powerful but also computationally involved. Quantization offers a way to reduce inference cost by lowering the precision of weights and activations, decreasing memory use and accelerating computation. While transformers can be more sensitive to precision loss than CNNs, applying quantization-aware training or mixed-precision strategies could make SAST+RGB more efficient without sacrificing much accuracy, enabling faster deployment on embedded devices.

Appendix	A
----------	----------

Task Description



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich



Task Description for a Master Thesis on

Exploiting Sparsity in Recurrent Transformer Architectures for Real-Time Object Detection using Event-Based Data

at the Department of Information Technology and
Electrical Engineering

for

Sebastian Heusinger
sheusinger@student.ethz.ch

Advisors: Jakub Mandula, jakub.mandula@pbl.ee.ethz.ch
Julian Moosmann, julian.moosmann@pbl.ee.ethz.ch
Dr. Philipp Mayer, mayerph@iis.ee.ethz.ch

Professor: Prof. Dr. John Lygeros

Handout Date: 01.03.2025
Due Date: 1.09.2025

Introduction

As recent events have demonstrated, the detection and tracking of drones in airspace have become a hot topic for national security, privacy, and safety [1]. The advent of small, highly maneuverable, and increasingly affordable drones has presented a unique set of challenges for existing surveillance and monitoring systems [2, 3, 4]. Radar or acoustic-based systems can struggle with detection due to the small drone size, while traditional camera-based monitoring systems struggle to keep up with the rapid movements against harsh background light conditions, especially in complex and dynamic environments. Event-based cameras, with their ability to capture changes in a scene with high temporal resolution, high dynamic range, and low latency, offer a promising solution to these challenges [5]. They are capable of detecting fast-moving objects like drones against a wide range of backgrounds and lighting conditions, making them an ideal technology for this purpose. Research into drone detection using event-based cameras is, therefore, a crucial area to explore. It holds the potential not only to enhance airspace security but also to advance the capabilities of surveillance technology in responding to the evolving dynamics of unmanned aerial vehicles.

Traditional computer vision systems rely on frame-based technology and use detectors such as YOLO [6, 7, 8]. Video sensors capture complete images at a given rate and resolution. These sequences of RGB frames are typically heavily redundant and thus inefficient to process. In contrast, event-based cameras (or silicon retinas) [9] have the potential to revolutionize vision sensors by emulating the biological retina. Instead of measuring pixel intensity at fixed intervals, these cameras detect and report significant changes in intensity as events. Event-based cameras offer multiple benefits, including high temporal resolution, low latency, extended dynamic range, low power consumption, and resistance to motion blur.

However, the event-driven nature of these vision sensors is rarely exploited in the current state-of-the-art research. State-of-the-art methods such as RVT [10] still use dense representation for operating on windows of events which partially defats the advantage of a sparse, event-based camera. More novel approaches such as GET [11] or SAST [12] attempt to partially exploit the

Project Goals

The goal of this thesis is to train, deploy, and benchmark a tracking system, that can exploit the event camera sparsity and demonstrate the performance benefits in a resource constraint device, such as the Jetson platform.

1. The exploration and deployment of various detection and tracking algorithms on the multi-camera system to be executed in real-time on the Jetson Orin Platform.
2. To ongoing development of the event-driven omnidirectional multi-camera system using the Sony IMX636 sensors DVS sensors.

3. Utilization of the bare IMX636 camera modules and interfacing directly with the Jetson Orin Platform.
4. Expansion of the platform beyond 2 sensors (if possible extension to up to 6 cameras)

The final system should be able to detect, classify, and track objects in a wide-angle camera view based on the event features. An analysis should be performed on the feasibility of extending the platform to 6, or possibly more cameras and if time allows, this should be implemented and demonstrated on the extended multi-sensor platform. Parameters such as inference time, speed, accuracy latency, and power consumption should be characterized and summarized in the final thesis report.

Tasks

The project will be split into four phases, as described below:

Phase 1 (Month 1)

1. Familiarization with Event-based detection, classification, and tracking algorithms, in particular, optimized for sparse event-data processing.
2. Literature research on DVS cameras, relevant drive concepts, and algorithms for object detection.
3. Familiarization with the existing multi-camera system.
4. Detailed project conceptualization, the definition of work packages, internal milestones, and project timeline.

Phase 2 (Month 2-4)

1. The implementation of SoTA event-driven detection, classification, and tracking algorithms, with a strong emphasis on sparse data processing [13].
2. The possible areas of exploration are classical detection algorithms, CNN-based algorithms, Transformer based algorithms [14], Sparse CNN-based algorithms [15], Graph-based algorithms [16, 17] and Recurrent networks [18, 19].
3. A baseline should be evaluated against the recently acquired Drone-detection dataset (which might need to be extended further with specific scenes eg. low light conditions or distant flying objects)

Phase 3 (Month 4-5)

1. Deployment of the algorithms on the Jetson platform and integration with the current 2-sensor expansion board and comparison to the EVK4 USB cameras in terms of throughput, latency, and power performance.
2. Optional extension to full 6 camera custom CSI interface board for Jetson and extension of the algorithm characterization using the full 360° view.

Phase 4 (Month 5-6)

1. System implementation and finetuning of the complete vision system.
2. Testing and characterization of the complete system (mechanical, electrical, software)
3. Finalize all the results, thesis wrap-up, and implementation of a full system demo.

Milestones

The following milestones need to be reached during the thesis:

- Background study finalized. ()
- SoTA algorithm implementation, analysis, and benchmark ()
- Deployment on the Jetson Orin platform ()
- Finalization full system evaluation.
- Final report and presentation. ()

Project Organization and Grading

During the thesis, students will gain experience in the independent solution of a technical-scientific problem by applying the acquired specialist and social skills.

The grade is based on the following: Student effort; thoroughness and learning curve; achieving qualitative and quantitative results with a scientific approach; supporting practical findings with theoretical background and literature investigations; final presentation and report; documentation and reproducibility. All theses include an oral presentation, and a written report and are graded. The report and presentation need to have publication-grade quality to achieve a good grade. Students are graded based on the official ITET grading form¹.

For students of IIS (Prof. Benini) a special grading scheme exists, please contact your supervisor for details there. Before starting, the project must be registered in myStudies and all required documents need to be handed in for archiving by PBL.

¹<https://ethz.ch/content/dam/ethz/special-interest/itet/departement/Studies/Forms/Grading%20Form.xlsx>

Laboratory Rules

The students agree to follow the lab rules set by PBL staff, for details please contact us. The most important points are:

- All ETH safety regulations need to be followed², in addition to ones given by PBL staff
- No device in the lab is used without introduction by your supervisor or PBL staff
- No device leaves the lab without being officially borrowed, this is done by PBL staff and needs your Legi.
- Any damage to devices or tools needs to be reported immediately to PBL staff.
- The Lab-desk is clean and free for others after you finish your task, or when you take longer breaks. All tools are correctly sorted into their drawers/cupboards when you leave

Weekly Report

There will be a weekly report/meeting held between the student and the assistants. The exact time and location of these meetings will be determined within the first week of the project to fit the students and the assistant's schedule. These meetings will be used to evaluate the status and document the progress of the project (required to be done by the student). Besides these regular meetings, additional meetings can be organized to address urgent issues as well. The weekly report, along with all other relevant documents (source code, datasheets, papers, etc), should be uploaded to a clouding service, such as Polybox, and shared with the assistants.

Project Plan

Within the first month of the project, you will be asked to prepare a project plan. This plan should identify the tasks to be performed during the project and sets deadlines for those tasks. The prepared plan will be a topic of discussion of the first week's meeting between you and your assistants. Note that the project plan should be updated constantly depending on the project's status.

Final Report and Paper

PDF copies of the final report written in English are to be turned in. Basic references will be provided by the supervisors by mail and at the meetings during the whole project, but the students are expected to add a considerable amount of their own literature research to the project ("state of the art").

²<https://ethz.ch/staffnet/en/3service/safety-security-health-environment/sicherheit-in-laboren-und-werkstaetten/laborsicherheit.html>

Final Presentation

There will be a presentation (15 min presentation and 5 min Q&A for BT/ST and 20 min presentation and 10 min Q&A for MT) at the end of this project in order to present your results to a wider audience. The exact date will be determined towards the end of the work.

References

Will be provided by the supervisors by mail and at the meetings during the whole project.

Place and Date Zurich, 20.02.2015 Signature Student Sebastian Heusinger

Bibliography

- [1] S. Calder. (2023) Five years on from gatwick drone disruption, what happened – and could it happen again? Accessed: 21-Sep-2024. [Online]. Available: <https://www.independent.co.uk/travel/news-and-advice/gatwick-drone-flights-cancelled-passengers-b2466653.html>
- [2] Z. Huang, S. Yang, M. Zhou, Z. Gong, A. Abusorrah, C. Lin, and Z. Huang, “Making accurate object detection at the edge: Review and new approach,” *Artificial Intelligence Review*, vol. 55, no. 3, pp. 2245–2274, 2022.
- [3] M. Wu, W. Xie, X. Shi, P. Shao, and Z. Shi, “Real-time drone detection using deep learning approach,” in *Machine Learning and Intelligent Communications: Third International Conference, MLICOM 2018, Hangzhou, China, July 6-8, 2018, Proceedings 3*. Springer, 2018, pp. 22–32.
- [4] G. Wang, Z. P. Bhat, Z. Jiang, Y.-W. Chen, D. Zha, A. C. Reyes, A. Niktash, G. Ulkar, E. Okman, X. Cai *et al.*, “Bed: A real-time object detection system for edge devices,” in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022, pp. 4994–4998.
- [5] G. Gallego, T. Delbrück, G. Orchard, C. Bartolozzi, B. Taba, A. Censi, S. Leutenegger, A. J. Davison, J. Conradt, K. Daniilidis *et al.*, “Event-based vision: A survey,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 44, no. 1, pp. 154–180, 2020.
- [6] C. Li, L. Li, H. Jiang, K. Weng, Y. Geng, L. Li, Z. Ke, Q. Li, M. Cheng, W. Nie *et al.*, “YOLOv6: A single-stage object detection framework for industrial applications,” *arXiv*, 2022.
- [7] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 7464–7475.
- [8] G. Jocher, A. Chaurasia, and J. Qiu, “YOLO by Ultralytics,” *github*, 2023, <https://github.com/ultralytics/ultralytics>. [Online]. Available: <https://github.com/ultralytics/ultralytics>

Bibliography

- [9] “Event-based Vision Evaluation Kits,” Jan. 2023, publication Title: PROPHESEE. [Online]. Available: <https://www.prophesee.ai/event-based-evaluation-kits/>
- [10] M. Gehrig and D. Scaramuzza, “Recurrent vision transformers for object detection with event cameras,” 2023. [Online]. Available: <https://arxiv.org/abs/2212.05598>
- [11] Y. Peng, Y. Zhang, Z. Xiong, X. Sun, and F. Wu, “Get: Group event transformer for event-based vision,” in *International Conference on Computer Vision (ICCV)*, 2023.
- [12] Y. Peng, H. Li, Y. Zhang, X. Sun, and F. Wu, “Scene adaptive sparse transformer for event-based object detection,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.01882>
- [13] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [14] A. Sabater, L. Montesano, and A. C. Murillo, “Event transformer. a sparse-aware solution for efficient event data processing,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 2677–2686.
- [15] N. Messikommer, D. Gehrig, A. Loquercio, and D. Scaramuzza, “Event-based asynchronous sparse convolutional networks,” 2020.
- [16] D. Gehrig and D. Scaramuzza, “Pushing the limits of asynchronous graph-based object detection with event cameras,” 2022.
- [17] S. Schaefer, D. Gehrig, and D. Scaramuzza, “Aegnn: Asynchronous event-based graph neural networks,” 2022.
- [18] L. Annamalai, V. Ramanathan, and C. S. Thakur, “Event-lstm: An unsupervised and asynchronous learning-based representation for event-based data,” *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4678–4685, 2022.
- [19] M. Cannici, M. Ciccone, A. Romanoni, and M. Matteucci, “A differentiable recurrent surface for asynchronous event-based data,” in *Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XX 16*. Springer, 2020, pp. 136–152.
- [20] J. Redmon and A. Farhadi, “YOLO9000: Better, Faster, Stronger,” in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 6517–6525.
- [21] —, “Yolov3: An incremental improvement,” *arXiv*, 2018.
- [22] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “Yolov4: Optimal speed and accuracy of object detection,” *arXiv*, 2020.
- [23] G. Jocher, “YOLOv5 by Ultralytics,” *github*, 2020. [Online]. Available: <https://github.com/ultralytics/yolov5>

Bibliography

- [24] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo *et al.*, “Segment anything,” *arXiv preprint arXiv:2304.02643*, 2023.
- [25] S. Singha and B. Aydin, “Automated drone detection using yolov4,” *Drones*, vol. 5, no. 3, p. 95, 2021.
- [26] B. Taha and A. Shoufan, “Machine learning-based drone detection and classification: State-of-the-art in research,” *IEEE access*, vol. 7, pp. 138 669–138 682, 2019.

Appendix	B
----------	----------

Declaration of Originality



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

- ☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies¹.
- ☒ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Multimodal and Sparse Recurrent Transformers
for Real-Time Object Detection

Authored by:

If the work was compiled in a group, the names of all authors are required.

Last name(s):

Hessinger

First name(s):

Sebastian

With my signature I confirm the following:

- I have adhered to the rules set out in the [Citation Guidelines](#).
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date

Zurich, 13.09.2025

Signature(s)

Hessinger

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

¹ For further information please consult the ETH Zurich websites, e.g. <https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html> and <https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html> (subject to change).

Bibliography

- [1] L. Kapustina, N. Izakova, E. Makovkina, and M. Khmelkov, “The global drone market: main development trends,” *SHS Web of Conferences*, vol. 129, p. 11004, 2021. [Online]. Available: <https://www.shs-conferences.org/10.1051/shsconf/202112911004>
- [2] S. Habibi, N. Ivaki, and J. Barata, “A Systematic Literature Review of Unmanned Aerial Vehicles for Healthcare and Emergency Services,” Apr. 2025, arXiv:2504.08834 [cs]. [Online]. Available: <http://arxiv.org/abs/2504.08834>
- [3] M. Bacco, A. Berton, A. Gotta, and L. Caviglione, “IEEE 802.15.4 Air-Ground UAV Communications in Smart Farming Scenarios,” *IEEE Communications Letters*, vol. 22, no. 9, pp. 1910–1913, Sep. 2018. [Online]. Available: <https://ieeexplore.ieee.org/document/8410566/>
- [4] S. Ri, J. Ye, N. Toyama, and N. Ogura, “Drone-based displacement measurement of infrastructures utilizing phase information,” *Nature Communications*, vol. 15, no. 1, p. 395, Jan. 2024, publisher: Nature Publishing Group. [Online]. Available: <https://www.nature.com/articles/s41467-023-44649-2>
- [5] M. Waibel, B. Keays, and F. Augugliaro, “Drone shows: Creative potential and best practices,” ETH Zurich, Tech. Rep., 2017, artwork Size: 18 p. Medium: application/pdf. [Online]. Available: <http://hdl.handle.net/20.500.11850/125498>
- [6] G. Kapoor, Y. S. Lee, R. Sikora, and S. Piramuthu, “Drone-based warehouse inventory management of perishables,” *International Journal of Production Economics*, vol. 278, p. 109437, Dec. 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925527324002949>
- [7] “‘Sustained’ drone attack closed Gatwick, airport says,” Feb. 2019. [Online]. Available: <https://www.bbc.com/news/business-47302902>
- [8] “‘Suspected drone’ disrupts Gatwick Airport flights,” May 2023. [Online]. Available: <https://www.bbc.com/news/uk-england-sussex-65591023>

Bibliography

- [9] Y. Zhi, Z. Fu, X. Sun, and J. Yu, “Security and Privacy Issues of UAV: A Survey,” *Mobile Networks and Applications*, vol. 25, no. 1, pp. 95–101, Feb. 2020. [Online]. Available: <https://doi.org/10.1007/s11036-018-1193-x>
- [10] “Poland downs drones after its airspace is violated,” *Reuters*, Sep. 2025. [Online]. Available: <https://www.reuters.com/business/aerospace-defense/poland-downs-drones-after-its-airspace-is-violated-2025-09-10/>
- [11] B. Barrett, “A Drone Tried to Disrupt the Power Grid. It Won’t Be the Last,” *Wired*, section: tags. [Online]. Available: <https://www.wired.com/story/drone-attack-power-substation-threat/>
- [12] W. Khawaja, M. Ezuma, V. Semkin, F. Erden, O. Ozdemir, and I. Guvenc, “A Survey on Detection, Classification, and Tracking of UAVs using Radar and Communications Systems,” Mar. 2025, arXiv:2402.05909 [eess]. [Online]. Available: <http://arxiv.org/abs/2402.05909>
- [13] Z. Shi, X. Chang, C. Yang, Z. Wu, and J. Wu, “An Acoustic-Based Surveillance System for Amateur Drones Detection and Localization,” *IEEE Transactions on Vehicular Technology*, vol. 69, no. 3, pp. 2731–2739, Mar. 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/8950116>
- [14] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, “YOLOv10: Real-Time End-to-End Object Detection,” Oct. 2024, arXiv:2405.14458 [cs]. [Online]. Available: <http://arxiv.org/abs/2405.14458>
- [15] R. Khanam and M. Hussain, “YOLOv11: An Overview of the Key Architectural Enhancements,” Oct. 2024, arXiv:2410.17725 [cs]. [Online]. Available: <http://arxiv.org/abs/2410.17725>
- [16] Q. Chen, X. Su, X. Zhang, J. Wang, J. Chen, Y. Shen, C. Han, Z. Chen, W. Xu, F. Li, S. Zhang, K. Yao, E. Ding, G. Zhang, and J. Wang, “LW-DETR: A Transformer Replacement to YOLO for Real-Time Detection,” Jun. 2024, arXiv:2406.03459 [cs]. [Online]. Available: <http://arxiv.org/abs/2406.03459>
- [17] C. Posch, T. Serrano-Gotarredona, B. Linares-Barranco, and T. Delbruck, “Retinomorph Event-Based Vision Sensors: Bioinspired Cameras With Spiking Output,” *Proceedings of the IEEE*, vol. 102, no. 10, pp. 1470–1484, Oct. 2014. [Online]. Available: <https://ieeexplore.ieee.org/document/6887319>
- [18] D. Zehnder, “Exploiting Sparsity in Recurrent Transformer Architectures for Real-Time Object Detection using Event-Based Data,” Ph.D. dissertation, ETH Zurich, Sep. 2024.
- [19] “Metavision® Sensors.” [Online]. Available: <https://www.prophesee.ai/event-based-sensors/>
- [20] “ace 2 a2A1920-160umPRO | USB 3.0 Camera | Basler | Basler AG.” [Online]. Available: <https://www.baslerweb.com/en/shop/a2a1920-160umpro/>

Bibliography

- [21] “3.4MP Low Light MIPI Camera | AR0330 Sensor Module.” [Online]. Available: https://www.e-consystems.com/ar0330-lowlight-cameramodule.asp?utm_source=chatgpt.com#key-features
- [22] “TRU COMPONENTS RS-OV7949-1818 Farb-Kamera-Modul 1 St. 5 V/DC (max) (L x B x H) 30 x 21 x 15 mm – Conrad Electronic Schweiz.” [Online]. Available: <https://www.conrad.ch/de/p/tru-components-rs-ov7949-1818-farb-kamera-modul-1-st-5-v-dc-max-l-x-b-x-h-30-x-21-x-15-mm-157.html>
- [23] N. Zubić, D. Gehrig, M. Gehrig, and D. Scaramuzza, “From Chaos Comes Order: Ordering Event Representations for Object Recognition and Detection,” Aug. 2023, arXiv:2304.13455 [cs]. [Online]. Available: <http://arxiv.org/abs/2304.13455>
- [24] Dharmaraj, “Convolutional Neural Networks (CNN) — Architectures Explained,” Jun. 2022. [Online]. Available: <https://medium.com/@draj0718/convolutional-neural-networks-cnn-architectures-explained-716fb197b243>
- [25] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” May 2016, arXiv:1506.02640 [cs]. [Online]. Available: <http://arxiv.org/abs/1506.02640>
- [26] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature Pyramid Networks for Object Detection,” Apr. 2017, arXiv:1612.03144 [cs]. [Online]. Available: <http://arxiv.org/abs/1612.03144>
- [27] J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement,” Apr. 2018, arXiv:1804.02767 [cs]. [Online]. Available: <http://arxiv.org/abs/1804.02767>
- [28] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, “YOLOX: Exceeding YOLO Series in 2021,” Aug. 2021, arXiv:2107.08430 [cs]. [Online]. Available: <http://arxiv.org/abs/2107.08430>
- [29] M. D. Zeiler and R. Fergus, “Visualizing and Understanding Convolutional Networks,” Nov. 2013, arXiv:1311.2901 [cs]. [Online]. Available: <http://arxiv.org/abs/1311.2901>
- [30] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, “Path Aggregation Network for Instance Segmentation,” Sep. 2018, arXiv:1803.01534 [cs]. [Online]. Available: <http://arxiv.org/abs/1803.01534>
- [31] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” Jun. 2021, arXiv:2010.11929 [cs]. [Online]. Available: <http://arxiv.org/abs/2010.11929>
- [32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” Aug. 2023, arXiv:1706.03762 [cs]. [Online]. Available: <http://arxiv.org/abs/1706.03762>

Bibliography

- [33] “Open Source Data Labeling.” [Online]. Available: <https://labelstud.io/>
- [34] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-End Object Detection with Transformers,” May 2020, arXiv:2005.12872 [cs]. [Online]. Available: <http://arxiv.org/abs/2005.12872>
- [35] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/6795963>
- [36] K. Cho, B. v. Merriënboer, D. Bahdanau, and Y. Bengio, “On the Properties of Neural Machine Translation: Encoder-Decoder Approaches,” Oct. 2014, arXiv:1409.1259 [cs]. [Online]. Available: <http://arxiv.org/abs/1409.1259>
- [37] fdeloche, “English: A diagram for a one-unit recurrent neural network (RNN). From bottom to top : input state, hidden state, output state. U, V, W are the weights of the network. Compressed diagram on the left and the unfold version of it on the right.” Jun. 2017. [Online]. Available: https://commons.wikimedia.org/wiki/File:Recurrent_neural_network_unfold.svg
- [38] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The Pascal Visual Object Classes (VOC) Challenge,” *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, Jun. 2010. [Online]. Available: <https://doi.org/10.1007/s11263-009-0275-4>
- [39] “COCO - Common Objects in Context.” [Online]. Available: <https://cocodataset.org/#detection-eval>
- [40] J. Ding, N. Xue, G.-S. Xia, X. Bai, W. Yang, M. Y. Yang, S. Belongie, J. Luo, M. Datcu, M. Pelillo, and L. Zhang, “Object Detection in Aerial Images: A Large-Scale Benchmark and Challenges,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7778–7796, Nov. 2022, arXiv:2102.12219 [cs]. [Online]. Available: <http://arxiv.org/abs/2102.12219>
- [41] A. M. Rekavandi, L. Xu, F. Boussaid, A.-K. Seghouane, S. Hoefs, and M. Bennamoun, “A Guide to Image and Video based Small Object Detection using Deep Learning : Case Study of Maritime Surveillance,” Jul. 2022, arXiv:2207.12926 [cs]. [Online]. Available: <http://arxiv.org/abs/2207.12926>
- [42] K. Yan, X. Wang, L. Lu, and R. M. Summers, “DeepLesion: Automated Deep Mining, Categorization and Detection of Significant Radiology Image Findings using Large-Scale Clinical Lesion Annotations,” Oct. 2017, arXiv:1710.01766 [cs]. [Online]. Available: <http://arxiv.org/abs/1710.01766>
- [43] G. Cheng, X. Yuan, X. Yao, K. Yan, Q. Zeng, X. Xie, and J. Han, “Towards Large-Scale Small Object Detection: Survey and Benchmarks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 11, pp. 13 467–13 488, Nov. 2023, conference Name: IEEE Transactions

Bibliography

- on Pattern Analysis and Machine Intelligence. [Online]. Available: <https://ieeexplore.ieee.org/document/10168277/?arnumber=10168277>
- [44] A. M. Rekavandi, S. Rashidi, F. Boussaid, S. Hoefs, E. Akbas, and M. bennamoun, “Transformers in Small Object Detection: A Benchmark and Survey of State-of-the-Art,” Sep. 2023, arXiv:2309.04902 [cs]. [Online]. Available: <http://arxiv.org/abs/2309.04902>
- [45] Q. Chen, J. Wang, C. Han, S. Zhang, Z. Li, X. Chen, J. Chen, X. Wang, S. Han, G. Zhang, H. Feng, K. Yao, J. Han, E. Ding, and J. Wang, “Group DETR v2: Strong Object Detector with Encoder-Decoder Pretraining,” Nov. 2022, arXiv:2211.03594 [cs]. [Online]. Available: <http://arxiv.org/abs/2211.03594>
- [46] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, “Deformable DETR: Deformable Transformers for End-to-End Object Detection,” Mar. 2021, arXiv:2010.04159 [cs]. [Online]. Available: <http://arxiv.org/abs/2010.04159>
- [47] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, “ImageNet Large Scale Visual Recognition Challenge,” Jan. 2015, arXiv:1409.0575 [cs]. [Online]. Available: <http://arxiv.org/abs/1409.0575>
- [48] M. Yaseen, “What is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector,” Aug. 2024, arXiv:2408.15857 [cs]. [Online]. Available: <http://arxiv.org/abs/2408.15857>
- [49] J. Mandula, J. Kühne, L. Pascarella, and M. Magno, “Towards Real-Time Fast Unmanned Aerial Vehicle Detection Using Dynamic Vision Sensors,” Mar. 2024, arXiv:2403.11875 [cs]. [Online]. Available: <http://arxiv.org/abs/2403.11875>
- [50] N. Chen, C. Xiao, Y. Dai, S. He, M. Li, and W. An, “Event-based Tiny Object Detection: A Benchmark Dataset and Baseline,” Jun. 2025, arXiv:2506.23575 [cs]. [Online]. Available: <http://arxiv.org/abs/2506.23575>
- [51] Z. Li, S. Niklaus, N. Snavely, and O. Wang, “Neural Scene Flow Fields for Space-Time View Synthesis of Dynamic Scenes,” Apr. 2021, arXiv:2011.13084 [cs]. [Online]. Available: <http://arxiv.org/abs/2011.13084>
- [52] S. Schaefer, D. Gehrig, and D. Scaramuzza, “AEGNN: Asynchronous Event-based Graph Neural Networks,” Nov. 2022, arXiv:2203.17149 [cs]. [Online]. Available: <http://arxiv.org/abs/2203.17149>
- [53] L. Cordone, B. Miramond, and P. Thierion, “Object Detection with Spiking Neural Networks on Automotive Event Data,” May 2022, arXiv:2205.04339 [cs]. [Online]. Available: <http://arxiv.org/abs/2205.04339>
- [54] N. Messikommer, D. Gehrig, A. Loquercio, and D. Scaramuzza, “Event-based Asynchronous Sparse Convolutional Networks,” Jul. 2020, arXiv:2003.09148 [cs]. [Online]. Available: <http://arxiv.org/abs/2003.09148>

Bibliography

- [55] M. Yao, H. Gao, G. Zhao, D. Wang, Y. Lin, Z. Yang, and G. Li, “Temporal-wise Attention Spiking Neural Networks for Event Streams Classification,” Jul. 2021, arXiv:2107.11711 [cs]. [Online]. Available: <http://arxiv.org/abs/2107.11711>
- [56] J. Zhang, B. Dong, H. Zhang, J. Ding, F. Heide, B. Yin, and X. Yang, “Spiking Transformers for Event-based Single Object Tracking,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. New Orleans, LA, USA: IEEE, Jun. 2022, pp. 8791–8800. [Online]. Available: <https://ieeexplore.ieee.org/document/9879994/>
- [57] M. Gehrig and D. Scaramuzza, “Recurrent Vision Transformers for Object Detection with Event Cameras,” May 2023, arXiv:2212.05598 [cs]. [Online]. Available: <http://arxiv.org/abs/2212.05598>
- [58] E. Perot, P. d. Tournemire, D. Nitti, J. Masci, and A. Sironi, “Learning to Detect Objects with a 1 Megapixel Event Camera,” Dec. 2020, arXiv:2009.13436 [cs]. [Online]. Available: <http://arxiv.org/abs/2009.13436>
- [59] Y. Peng, H. Li, Y. Zhang, X. Sun, and F. Wu, “Scene Adaptive Sparse Transformer for Event-based Object Detection,” Apr. 2024, arXiv:2404.01882 [cs]. [Online]. Available: <http://arxiv.org/abs/2404.01882>
- [60] G. Bellec, F. Scherr, A. Subramoney, E. Hajek, D. Salaj, R. Legenstein, and W. Maass, “A solution to the learning dilemma for recurrent networks of spiking neurons,” *Nature Communications*, vol. 11, no. 1, p. 3625, Jul. 2020. [Online]. Available: <https://doi.org/10.1038/s41467-020-17236-y>
- [61] J. H. Lee, T. Delbruck, and M. Pfeiffer, “Training Deep Spiking Neural Networks using Backpropagation,” Aug. 2016, arXiv:1608.08782 [cs]. [Online]. Available: <http://arxiv.org/abs/1608.08782>
- [62] C. Lee, S. S. Sarwar, P. Panda, G. Srinivasan, and K. Roy, “Enabling Spike-based Backpropagation for Training Deep Neural Network Architectures,” *Frontiers in Neuroscience*, vol. 14, p. 119, Feb. 2020, arXiv:1903.06379 [cs]. [Online]. Available: <http://arxiv.org/abs/1903.06379>
- [63] Q. Su, Y. Chou, Y. Hu, J. Li, S. Mei, Z. Zhang, and G. Li, “Deep Directly-Trained Spiking Neural Networks for Object Detection,” Jul. 2023, arXiv:2307.11411 [cs]. [Online]. Available: <http://arxiv.org/abs/2307.11411>
- [64] G. Magrini, F. Becattini, P. Pala, A. D. Bimbo, and A. Porta, “Neuromorphic Drone Detection: an Event-RGB Multimodal Approach,” Sep. 2024, arXiv:2409.16099 [cs]. [Online]. Available: <http://arxiv.org/abs/2409.16099>
- [65] H. Zhang, X. Wang, C. Xu, X. Wang, F. Xu, H. Yu, L. Yu, and W. Yang, “Frequency-Adaptive Low-Latency Object Detection Using Events and Frames,” Dec. 2024, arXiv:2412.04149 [cs]. [Online]. Available: <http://arxiv.org/abs/2412.04149>

Bibliography

- [66] G. Magrini, N. Marini, F. Becattini, L. Berlincioni, N. Biondi, P. Pala, and A. D. Bimbo, “FRED: The Florence RGB-Event Drone Dataset,” Jun. 2025, arXiv:2506.05163 [cs]. [Online]. Available: <http://arxiv.org/abs/2506.05163>
- [67] “COCO - Common Objects in Context.” [Online]. Available: <https://cocodataset.org/#detection-eval>
- [68] “The HDF5® Library & File Format.” [Online]. Available: <https://www.hdfgroup.org/solutions/hdf5/>
- [69] “numpy.savez_compressed — NumPy v2.4.dev0 Manual.” [Online]. Available: https://numpy.org/devdocs/reference/generated/numpy.savez_compressed.html#numpy.savez_compressed
- [70] “Command-Line Programs — NVIDIA TensorRT Documentation.” [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/latest/reference/command-line-programs.html>
- [71] “ONNX Runtime | Home.” [Online]. Available: <https://onnxruntime.ai/>
- [72] “ultralytics/thop,” Sep. 2025, original-date: 2024-05-31T17:26:59Z. [Online]. Available: <https://github.com/ultralytics/thop>